

博弈搜索

祝恩
计算机学院
国防科技大学

学习内容

1 博弈问题：

如何应对巨大状态空间？
博弈问题的形式化。

2 博弈中的优化决策

Minimax算法

多人博弈时的最优决策

3 α - β 剪枝和行棋排序

4 不完美的实时决策

5 随机博弈



能够给出博弈的形式化描述

对抗搜索

1 博弈问题



经济学中的博弈论

- 在**经济学**中，只要每个Agent对其他Agent的影响是显著的，无论是合作环境还是竞争环境，**博弈论**都将多Agent环境看成是**博弈**。包含非常多Agent的环境被视为**经济系统**，而不再是视为博弈。
- 在**人工智能**中，**最常见**的一类特殊的博弈是有完整信息的、确定性的、轮流行动的、两个游戏者的**零和游戏**（如国际象棋）。Agent之间的效用函数是对立的。

一类特殊的博弈

- 最常见的一类特殊类型的博弈：
 - Two-player, turn-taking
 - fully observable, deterministic
 - zero-sum (零和)
 - time-constrained

一些典型的博弈游戏

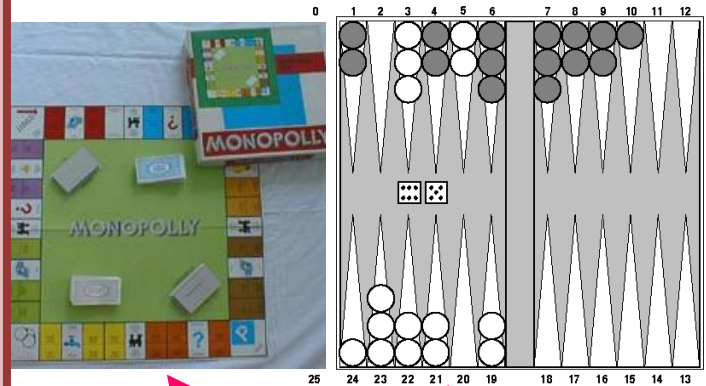
国际象棋
chess

西洋跳棋
checkers

黑白棋/奥赛罗
othello

大富翁
monopoly

西洋双陆棋
backgammon



perfect information

Chess, Checkers,
Go, Othello, Grundy

Backgammon
Monopoly

imperfect information

Bridge, Poker,
Scrabble, Nuclear war

deterministic

chance

为什么研究博弈

□ 为什么研究博弈

- 娱乐需求：下棋、桥牌、 ...
- 是AI的经典研究问题
- 学会如何应对其他Agent：社会、经济、 ...
- 博弈问题的通常有巨大搜索空间，求解困难

应对巨大的状态空间

- Many game programs are based on
 - alpha-beta + iterative deepening + huge databases + ...
- The methods are general, but their implementation is dramatically improved by many specifically tuned-up enhancements (e.g., the **evaluation functions**).
- Go (围棋) has too high a branching factor for search techniques. Go software must rely on **huge databases and pattern-recognition** techniques.
- **Search** is very important.

形式化博弈问题

- 考虑两人博弈：MAX和MIN，（为什么这样命名）。MAX先行，两人轮流出招，直到游戏结束。给胜者加分，给败者罚分。
- 博弈可以形式化成一类搜索问题，含有特定的组成部分。



形式化博弈问题

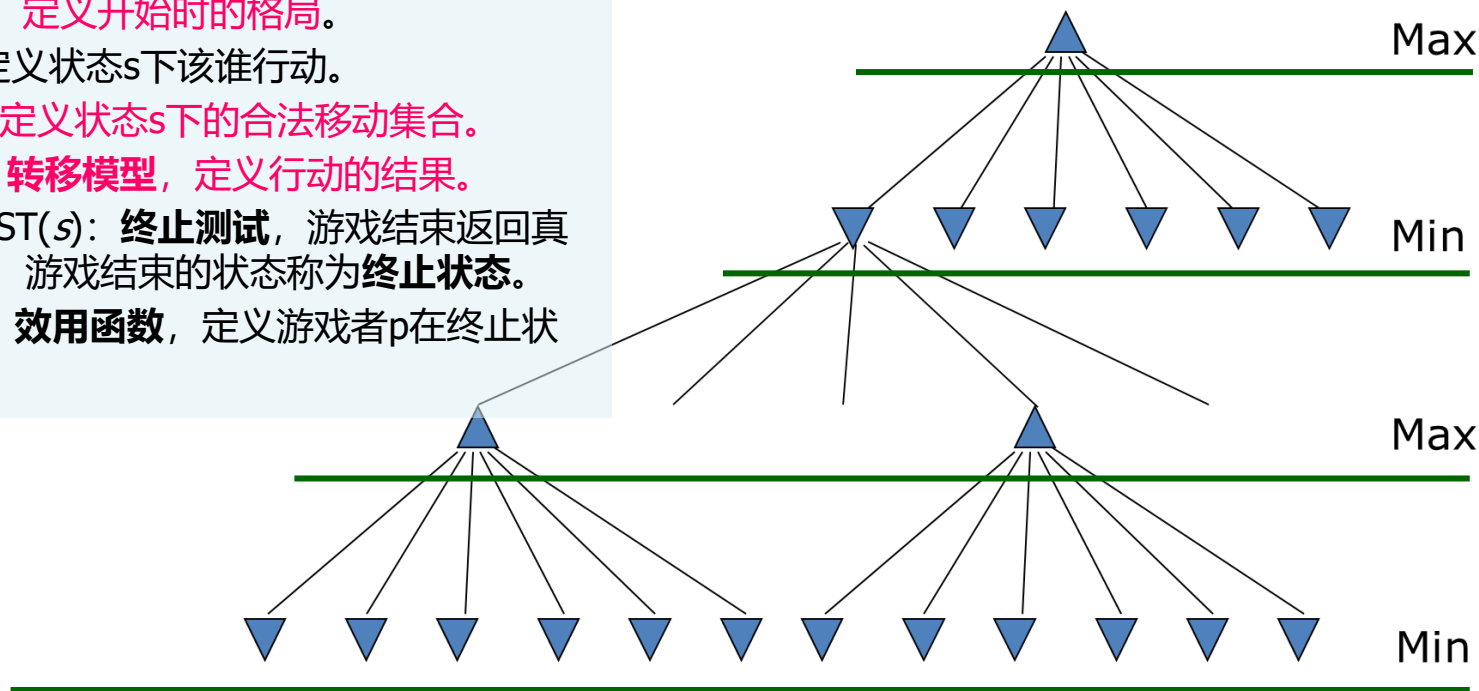
序号	形式化	解释
1	S_0	初始状态 ，定义开始时的格局。
2	$PLAYER(s)$	定义状态 s 下该谁行动。
3	$ACTIONS(s)$	定义状态 s 下的合法移动集合。
4	$RESULT(s,a)$	转移模型 ，定义行动的结果。
5	$TERMINAL-TEST(s)$	终止测试 ，游戏结束返回真，否则返回假。 游戏结束的状态称为 终止状态 。
6	$UTILITY(s,p)$	效用函数 ，定义游戏者 p 在终止状态 s 下的数值。 零和博弈 是指所有棋手的收益之和在每个棋局实例中都相同。国际象棋中是 $0+1$ ， $1+0$ 或 $\frac{1}{2}+\frac{1}{2}$ 。

初始状态、ACTIONS函数和RESULT函数定义了游戏的**博弈树**——其中结点是状态，边是移动。

博弈树(Game Tree)

- 由初始状态、ACTIONS函数和RESULT函数确定
- 结点是状态
- 边是可能的走子

1. S_0 : **初始状态**, 定义开始时的格局。
2. $\text{PLAYER}(s)$: 定义状态 s 下该谁行动。
3. $\text{ACTIONS}(s)$: 定义状态 s 下的合法移动集合。
4. $\text{RESULT}(s, a)$: **转移模型**, 定义行动的结果。
5. $\text{TERMINAL-TEST}(s)$: **终止测试**, 游戏结束返回真, 否则返回假。游戏结束的状态称为**终止状态**。
6. $\text{UTILITY}(s, p)$: **效用函数**, 定义游戏者 p 在终止状态 s 下的数值。



能够解释极小极大算法的原理
能够用极小极大法分析博弈结果

minimax算法
多人博弈时的最优决策

2 博弈中的优化决策



博弈中的搜索

- 对手Agent (Min)和我们的Agent (Max)对抗
- Agent考虑自己的行动时必须考虑对手的可能行动
- 对手的行动未知
- 决策的时间和空间资源有限
- 状态空间巨大，有限时间内只能考虑很小的一部分

暂时不考虑时间空间限制

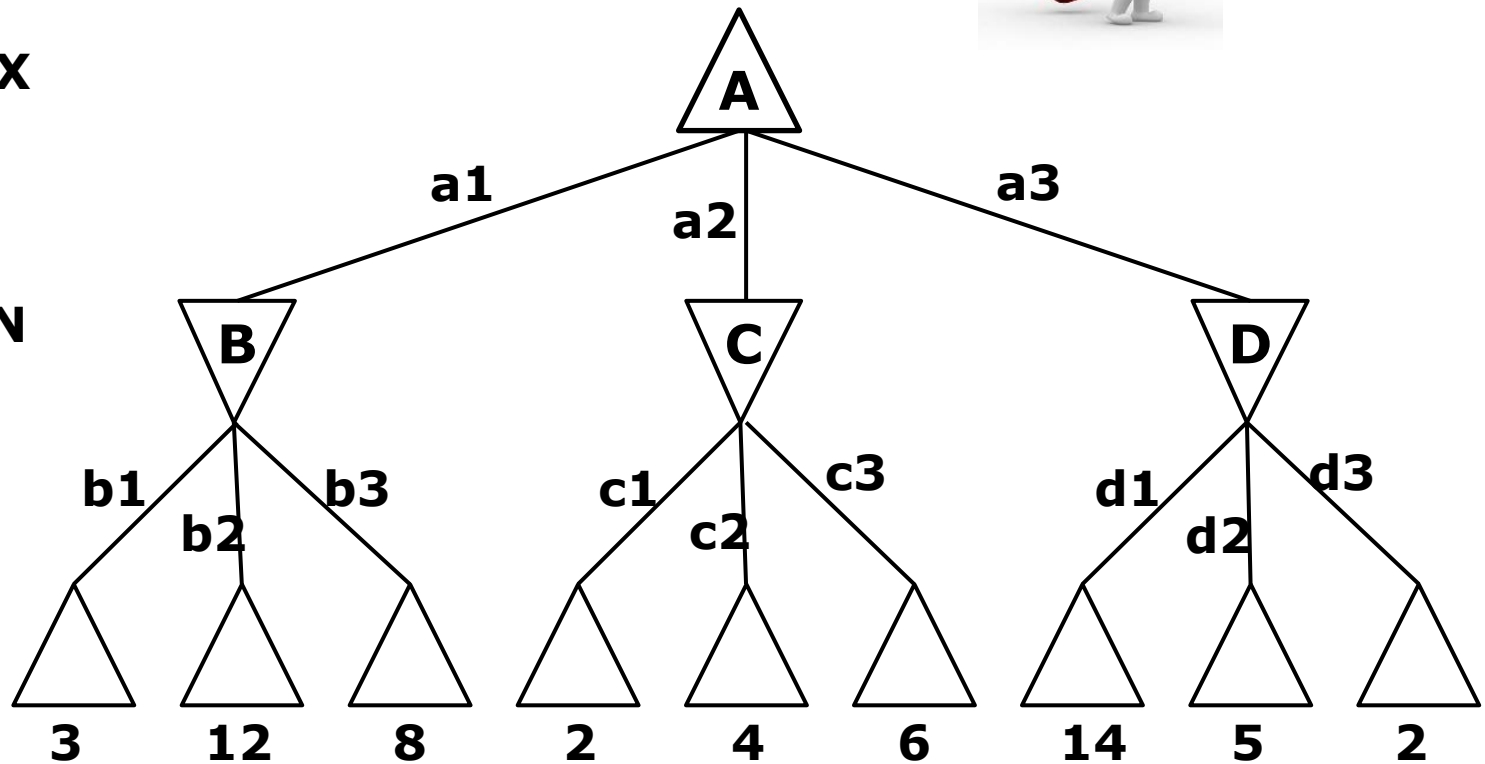
MAX的最优决策

□ MAX选择哪一步? a1/a2/a3



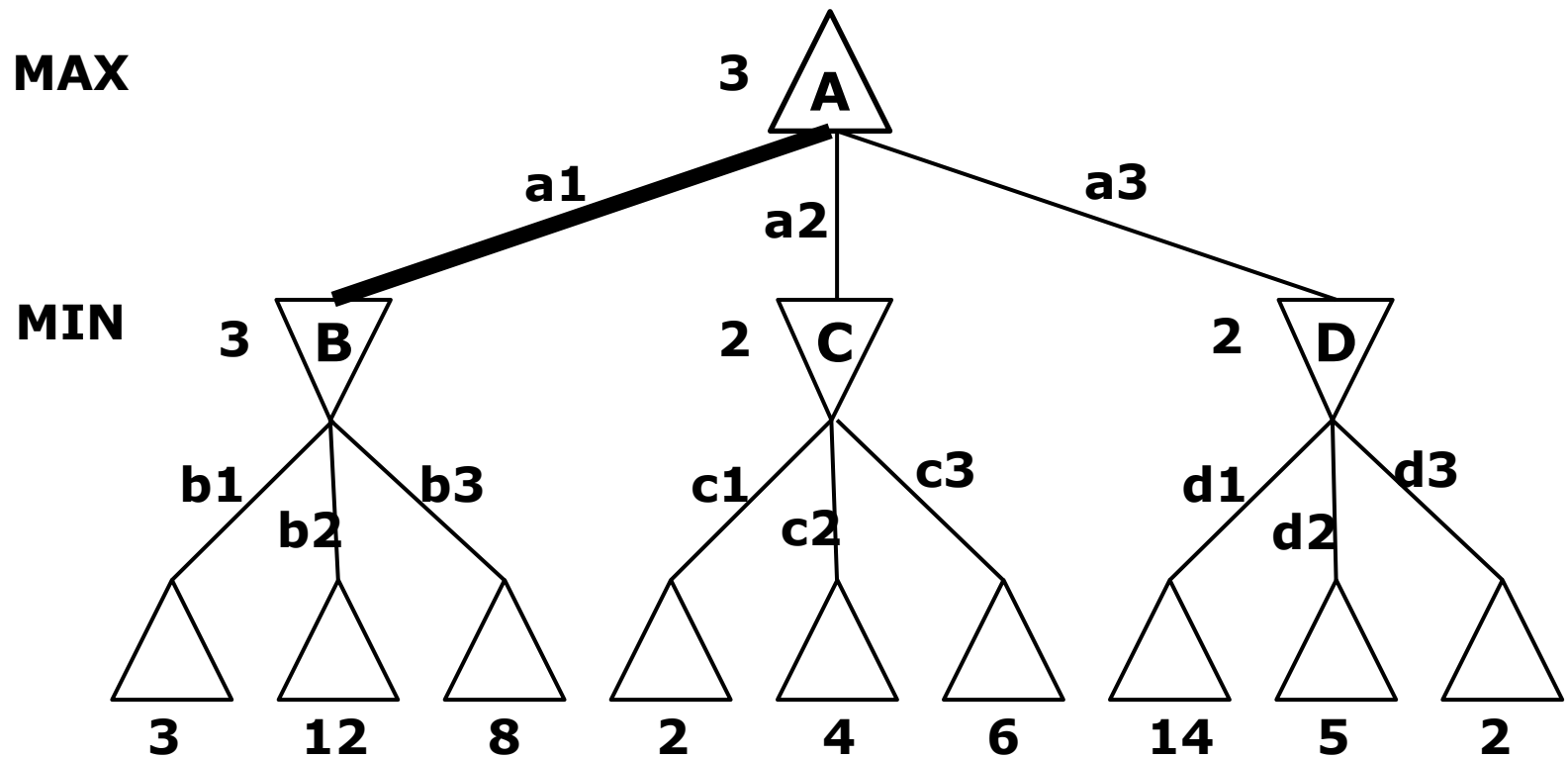
MAX

MIN



假设构造从当前状态到所有可能终止状态的博弈

MAX的最优决策

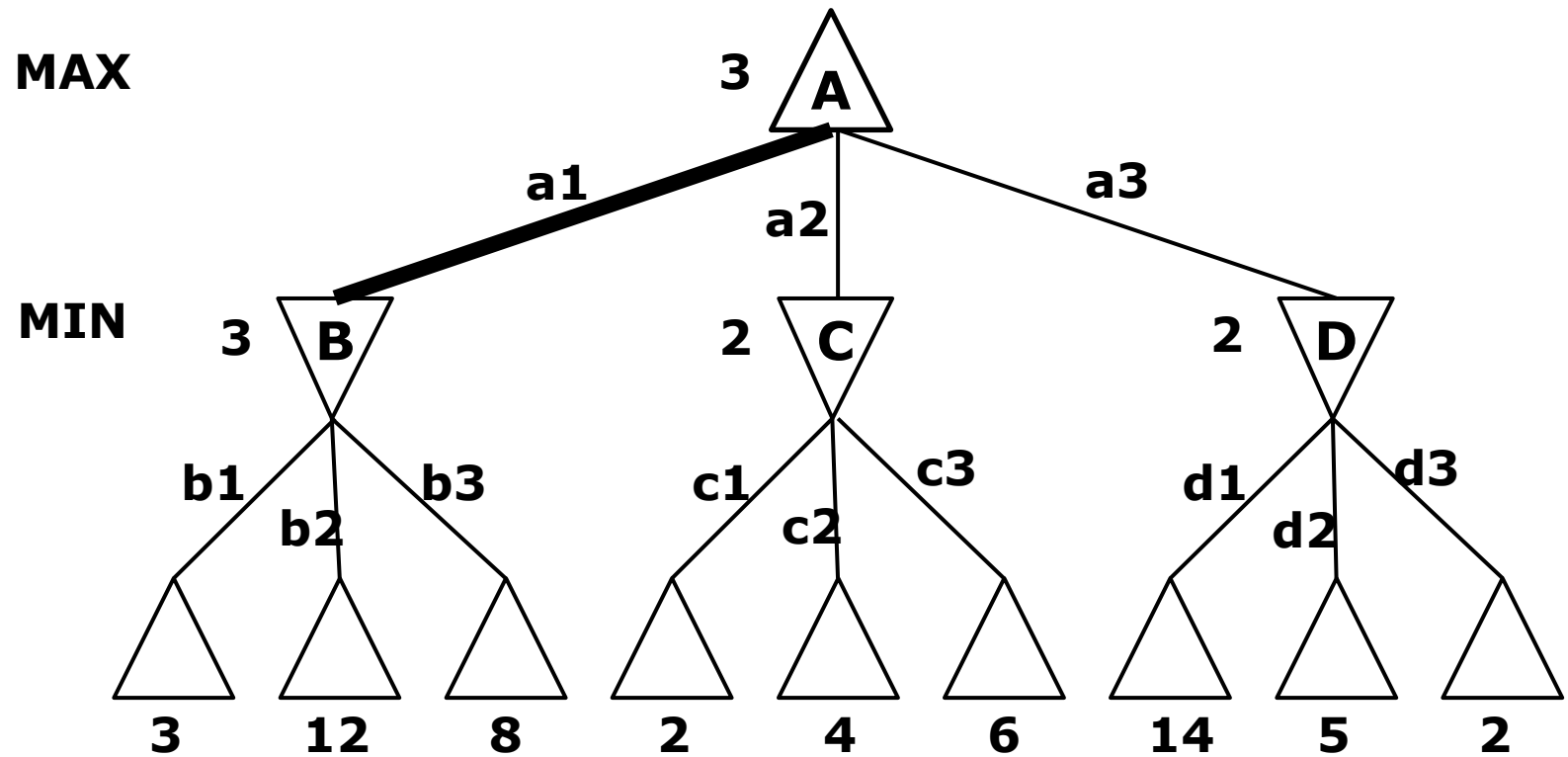


$\text{MINIMAX}(s) =$



写出计算s的minimax值的公式

MAX的最优决策



$\text{MINIMAX}(s) =$

$$\begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

MiniMax算法

function MINIMAX-DECISION(*state*) *returns an action*
return $\arg \max_{a \in \text{ACTIONS}(s)} \text{MIN-VALUE}(\text{RESULT}(state, a))$

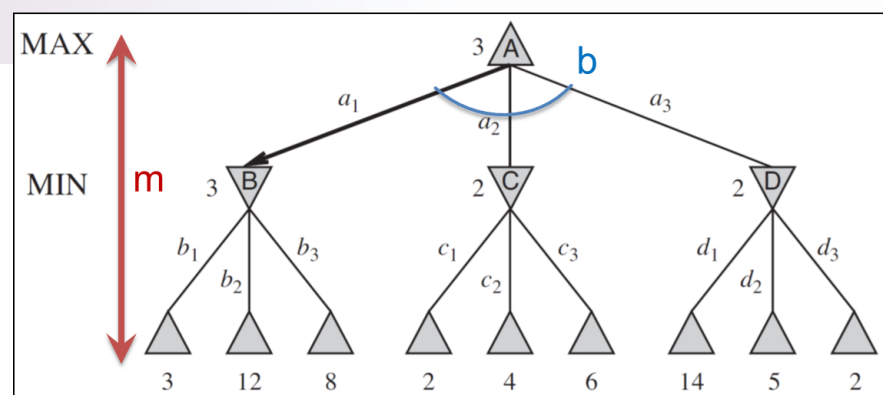
function MAX-VALUE(*state*) *returns a utility value*
if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
 $v \leftarrow -\infty$
for each *a* **in** ACTIONS(*state*) **do**
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a)))$
return *v*

function MIN-VALUE(*state*) *returns a utility value*
if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
 $v \leftarrow \infty$
for each *a* **in** ACTIONS(*state*) **do**
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a)))$
return *v*

MINIMAX(*s*) =

$$\begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

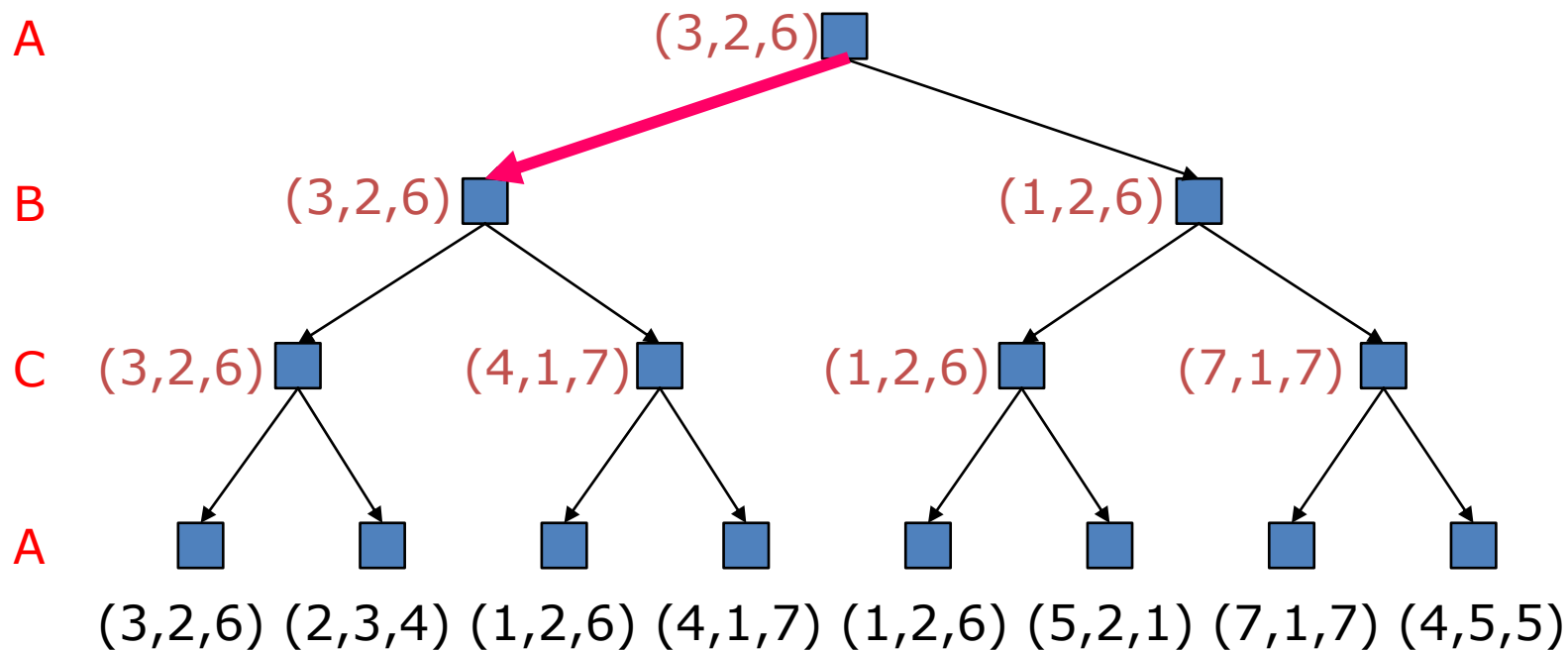
MiniMax算法评价



- 本质上是一种深度优先策略
 - 深度：树的最大深度为 m
 - 分支因子：每个结点可考虑的行棋选择有 b 种选择
- 时间复杂度： $O(b^m)$
- 空间复杂度
 - 一次性生成所有后继： $O(bm)$
 - 每次生成一个后继： $O(m)$
- 对于真实游戏极小极大算法不实用，但可作为研究和设计其它算法的基础

多人博弈中的最优决策

- 若博弈有三个人A, B和C参与, 则每个结点都与一个向量 $\langle v_A, v_B, v_C \rangle$ 相关联。对于终止状态, 这个向量代表着从每个人角度出发得到的状态效用值; 最简单的实现方法就是让函数UTILITY返回一个**效用值向量**。对于非终止状态, 它的值是它选择的后继结点效用值向量。



多人博弈中的联盟

- 多人博弈涉及选手之间出现**联盟**的情况。假设A和B相对较弱，而C比较强。对于A和B而言，它们一起进攻C比等C逐个消灭它们要好，这通常是最优策略。这样，博弈从纯自私的行为变成合作。
- 联盟不断建立或者解散。

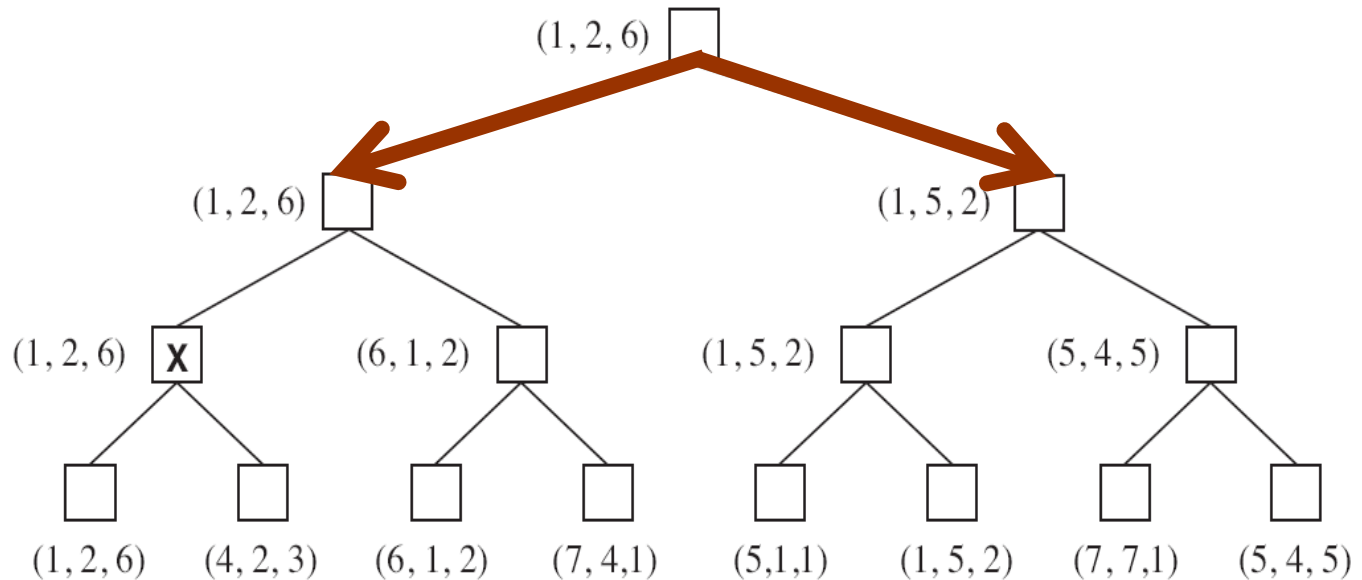
to move

A

B

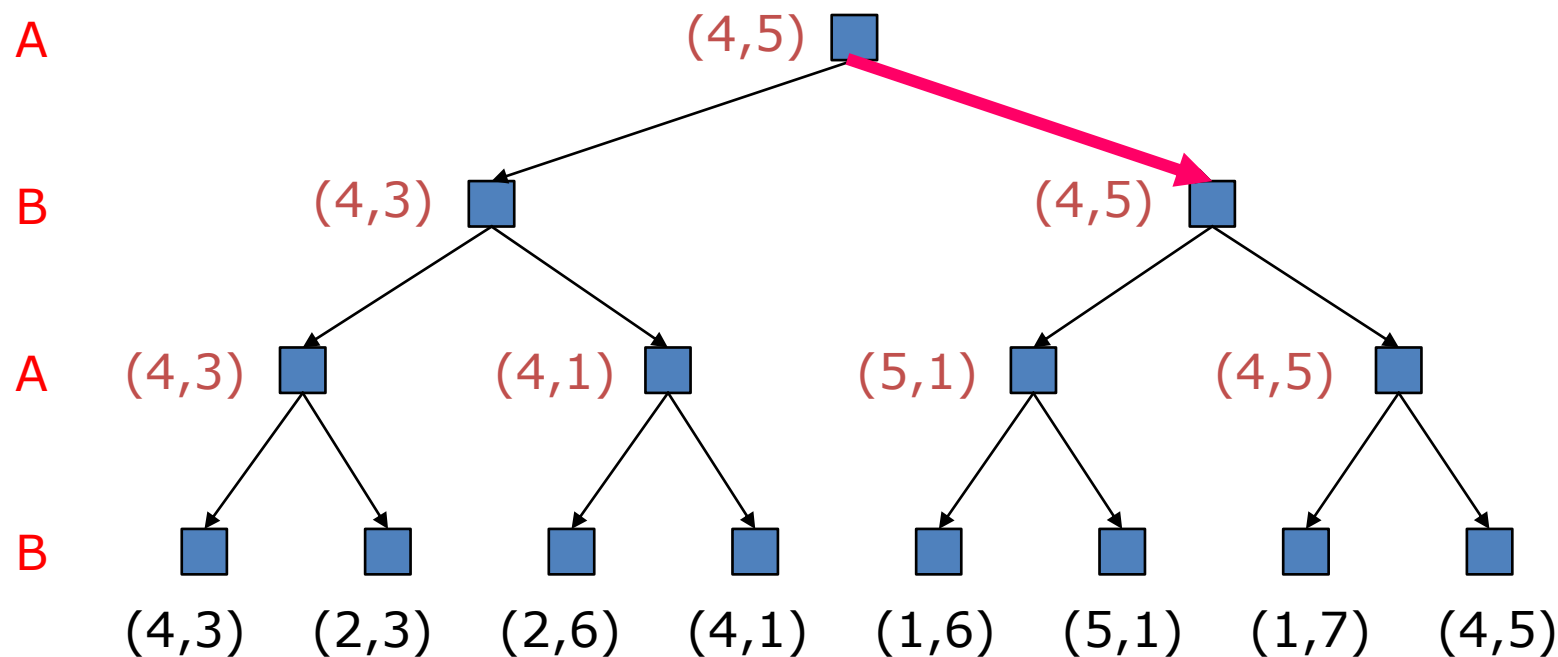
C

A



非零和的两人博弈也可能发生合作

- 如果博弈是**非零和**的，合作也可能发生在两人博弈中。例如，假设终止状态的效用值向量是 $\langle v_A=10, v_B=10 \rangle$ ，并且10对于两个选手都是最高的可能效用值。



能够解释剪枝算法的原理

能够根据剪枝算法完成对博弈树的剪枝

能够分析 α - β 值的传递情况，写出具体实例的传递值和返回值

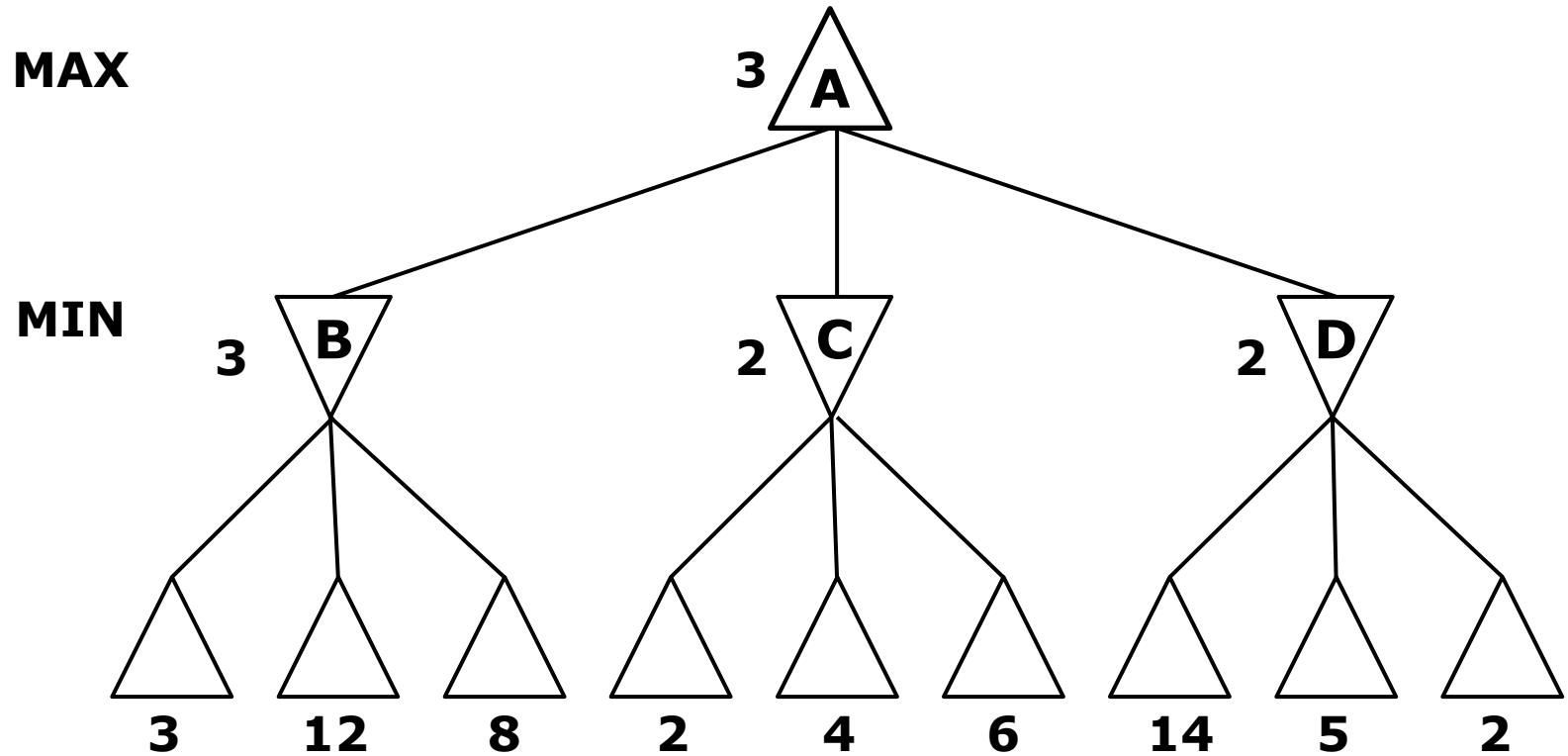
剪枝算法

行棋排序

3 α - β 剪枝与行棋排序



MiniMax可以改进

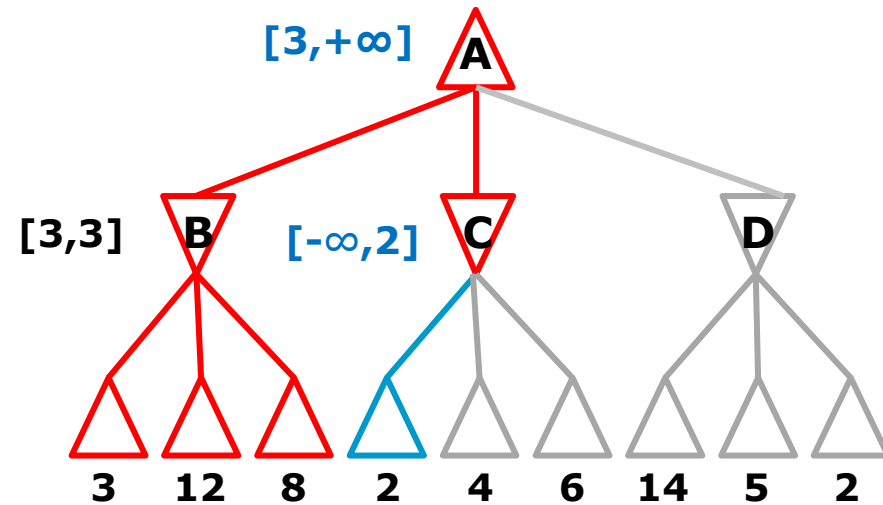
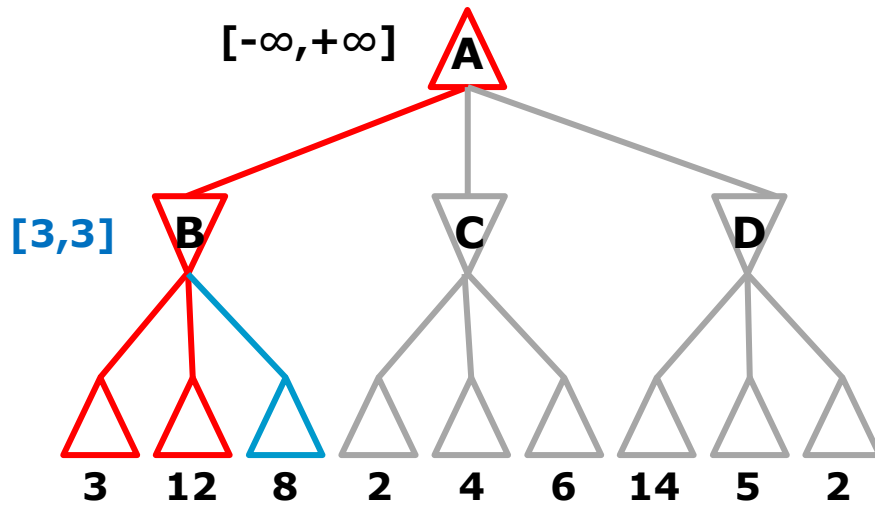
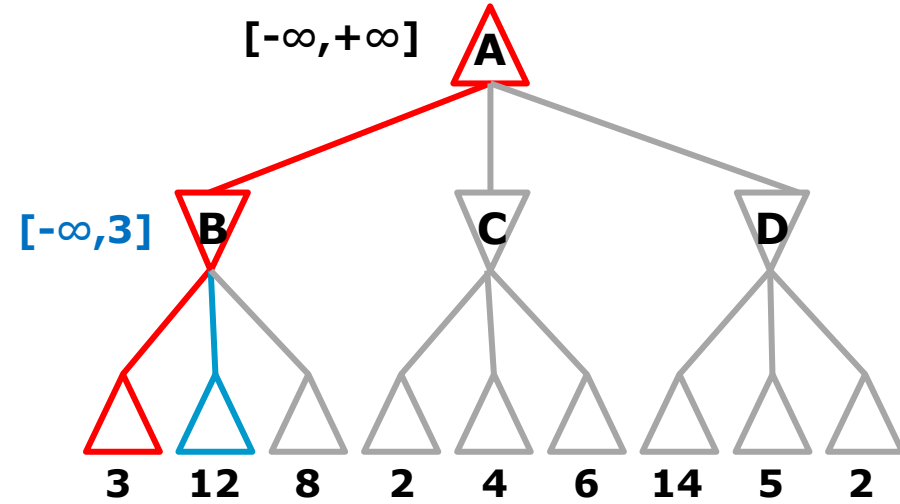
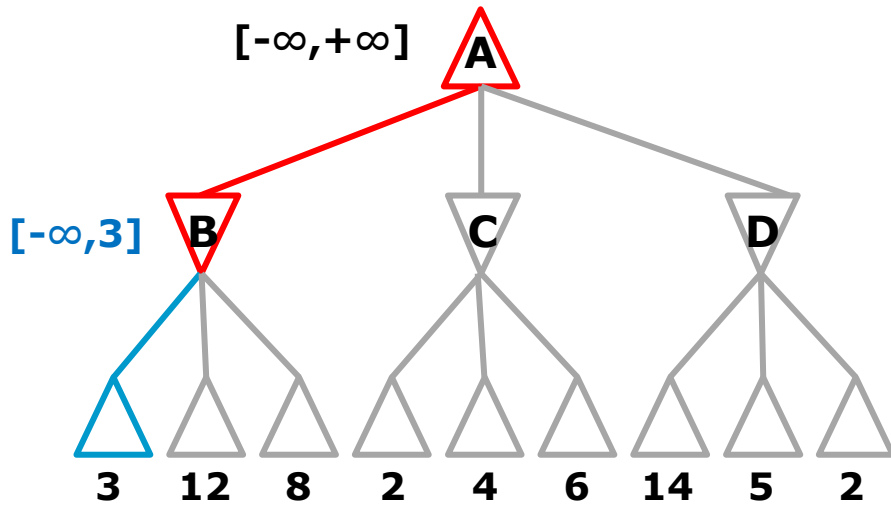


$$\text{MINIMAX}(\text{root}) = \max(\min(3, 12, 8), \min(2, 4, 6), \min(14, 5, 2))$$



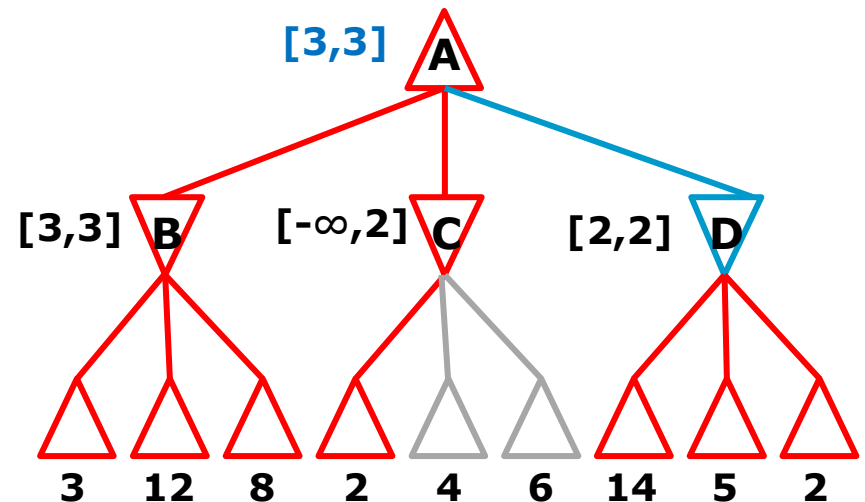
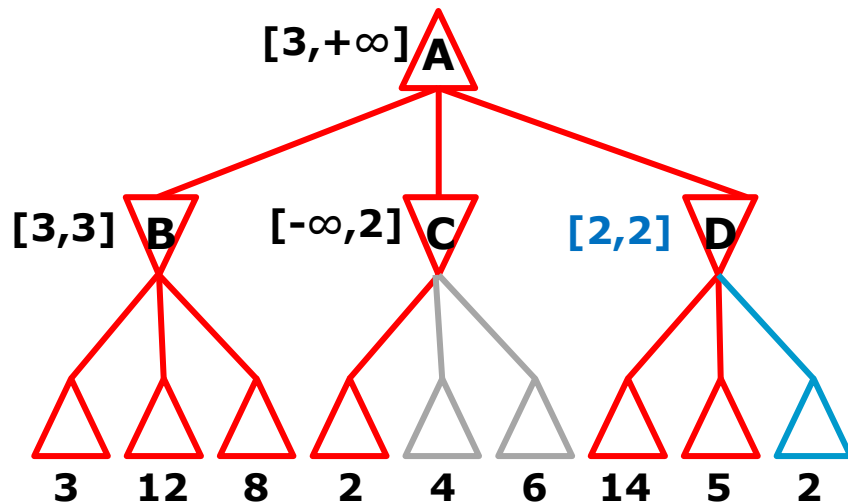
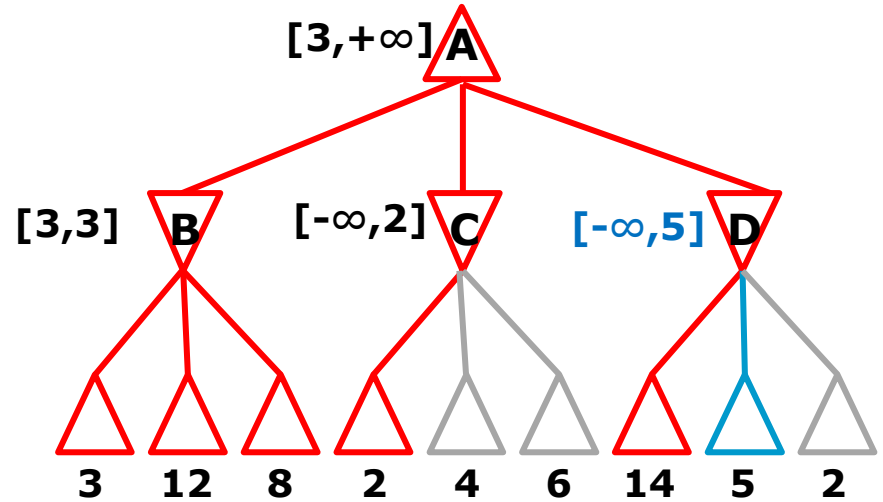
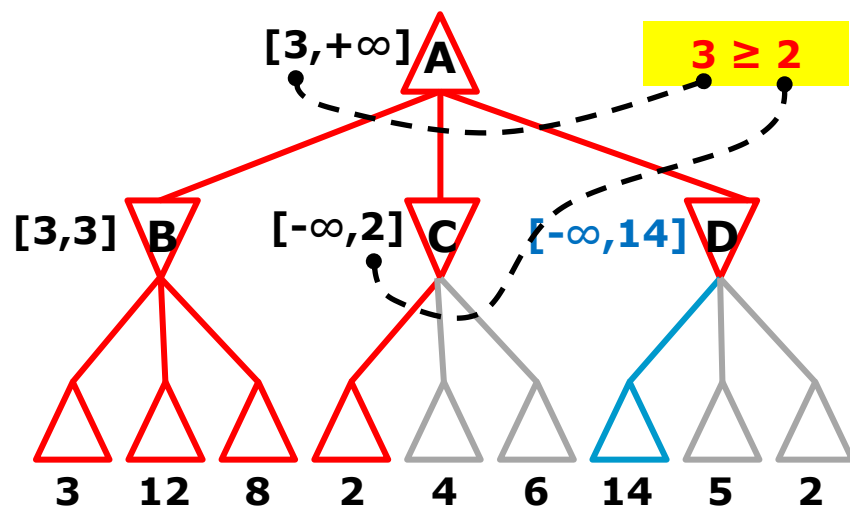
哪些结点不必搜索？
即 $\max()$ 的值与哪些值无关？

随着搜索进行，分析结点的值的范围



each node is marked with a **range** of its value

随着搜索进行，分析结点的值的范围



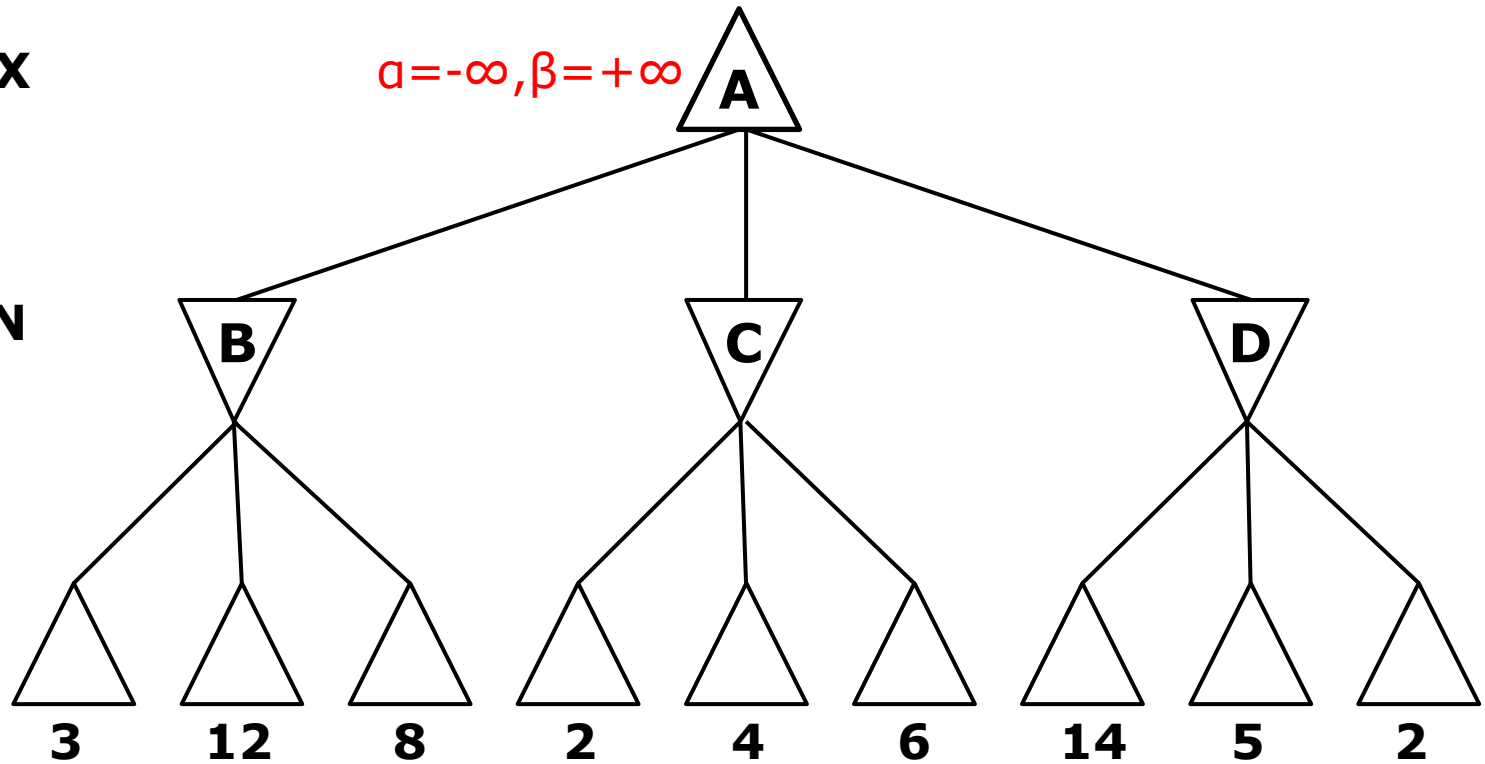
each node is marked with a **range** of its value

α - β 剪枝

MAX

$\alpha = -\infty, \beta = +\infty$

MIN

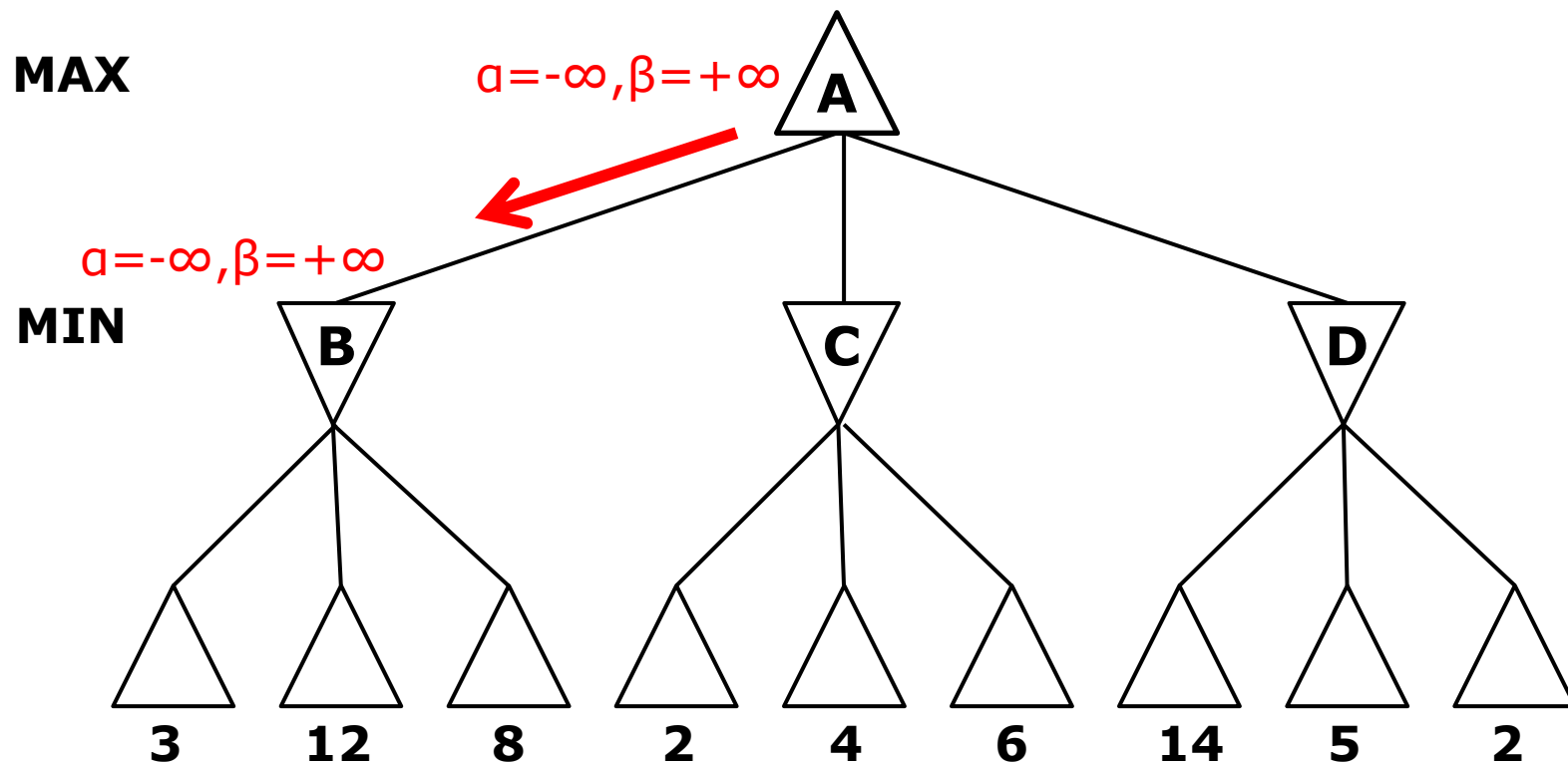


α = 到目前为止路径上**MAX**获得的最佳值（即极大值）

β = 到目前为止路径上**MIN**获得的最佳值（即极小值）

v = 当前结点所搜索过的子结点中的最佳值

α - β 剪枝

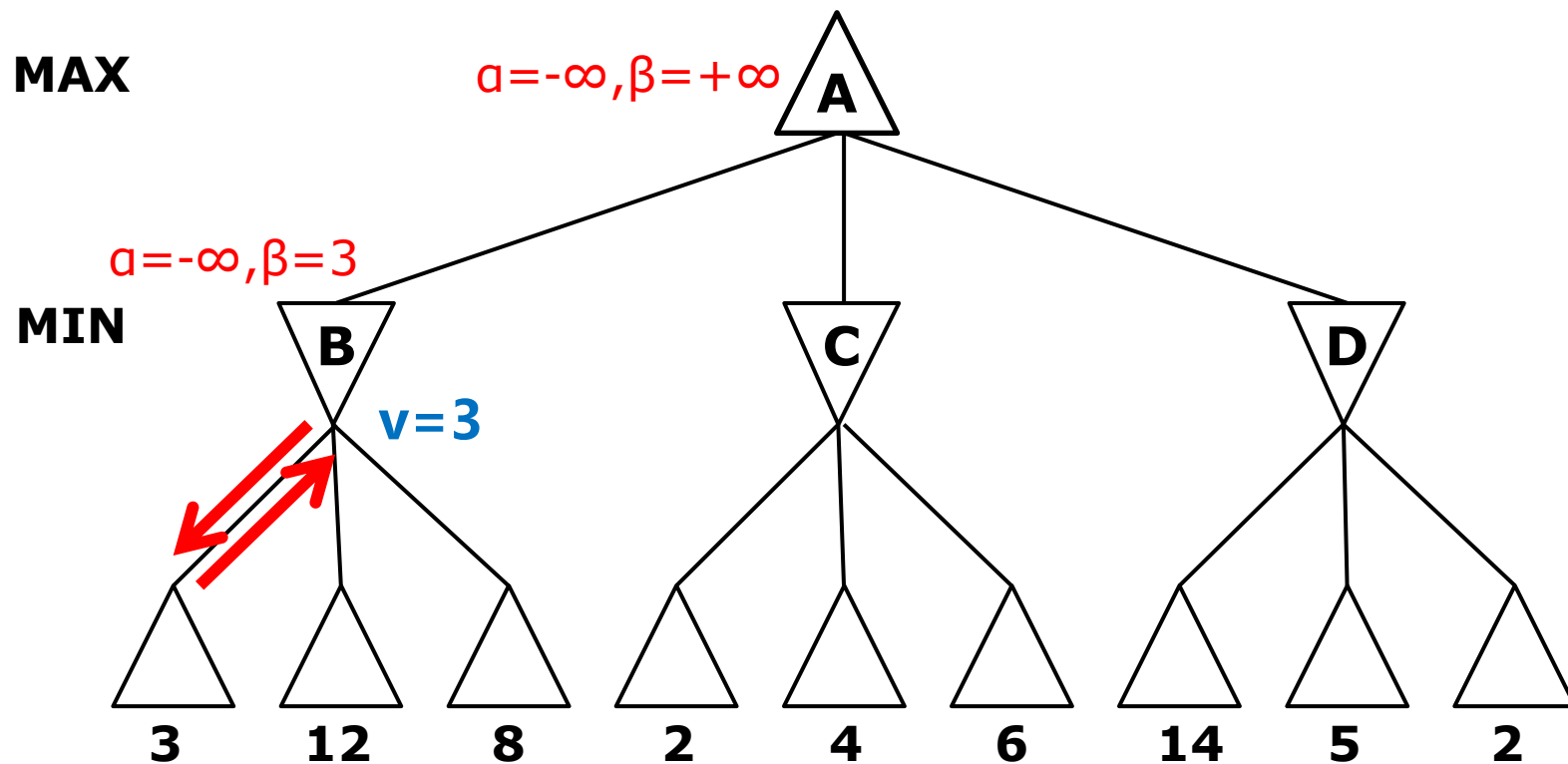


α = 到目前为止路径上MAX获得的最佳值（即极大值）

β = 到目前为止路径上MIN获得的最佳值（即极小值）

v = 当前结点所搜索过的子结点中的最佳值

α - β 剪枝

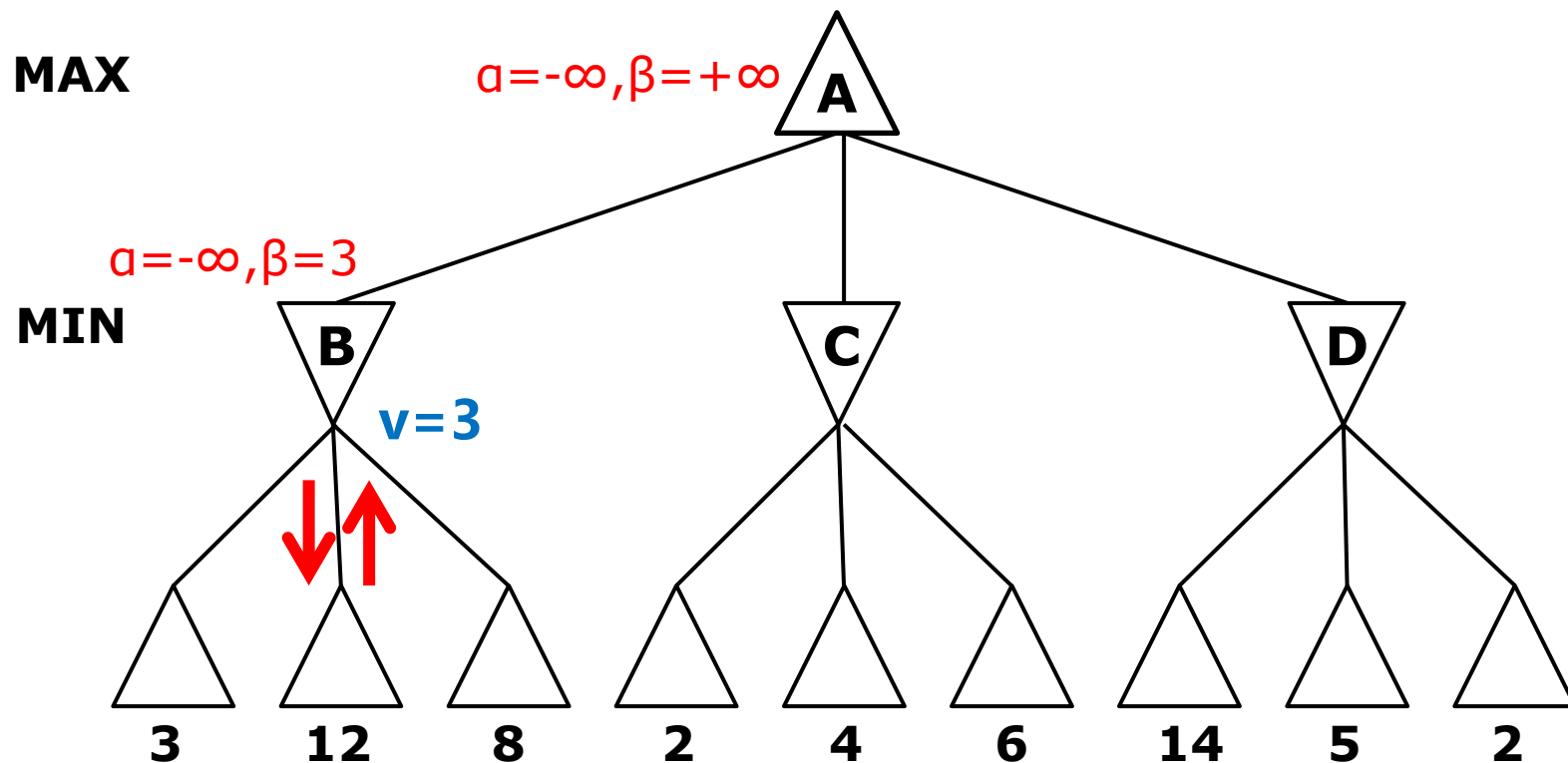


α = 到目前为止路径上**MAX**获得的最佳值（即极大值）

β = 到目前为止路径上**MIN**获得的最佳值（即极小值）

v = 当前结点所搜索过的子结点中的最佳值

α - β 剪枝

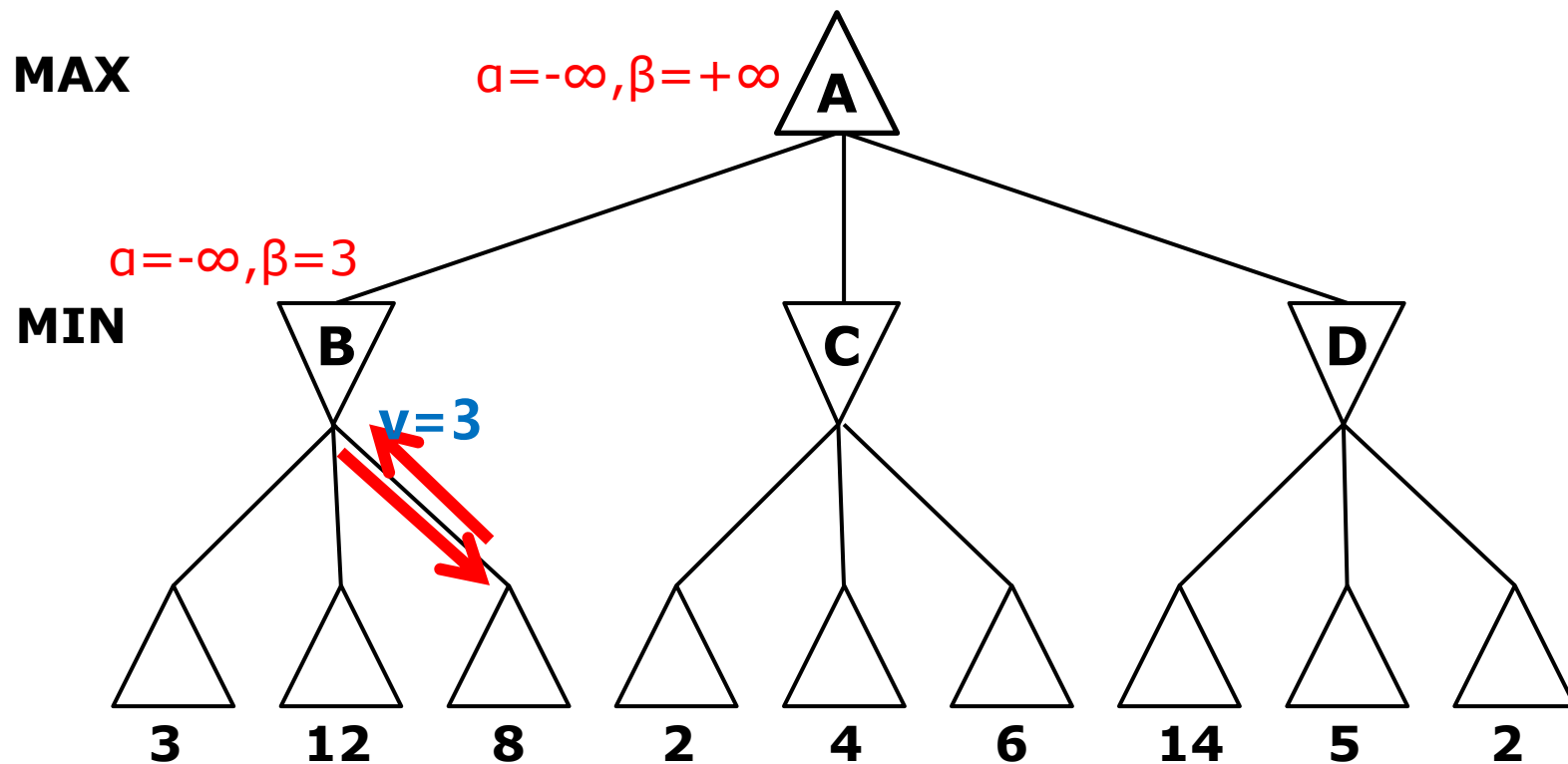


α = 到目前为止路径上MAX获得的最佳值（即极大值）

β = 到目前为止路径上MIN获得的最佳值（即极小值）

v = 当前结点所搜索过的子结点中的最佳值

α - β 剪枝

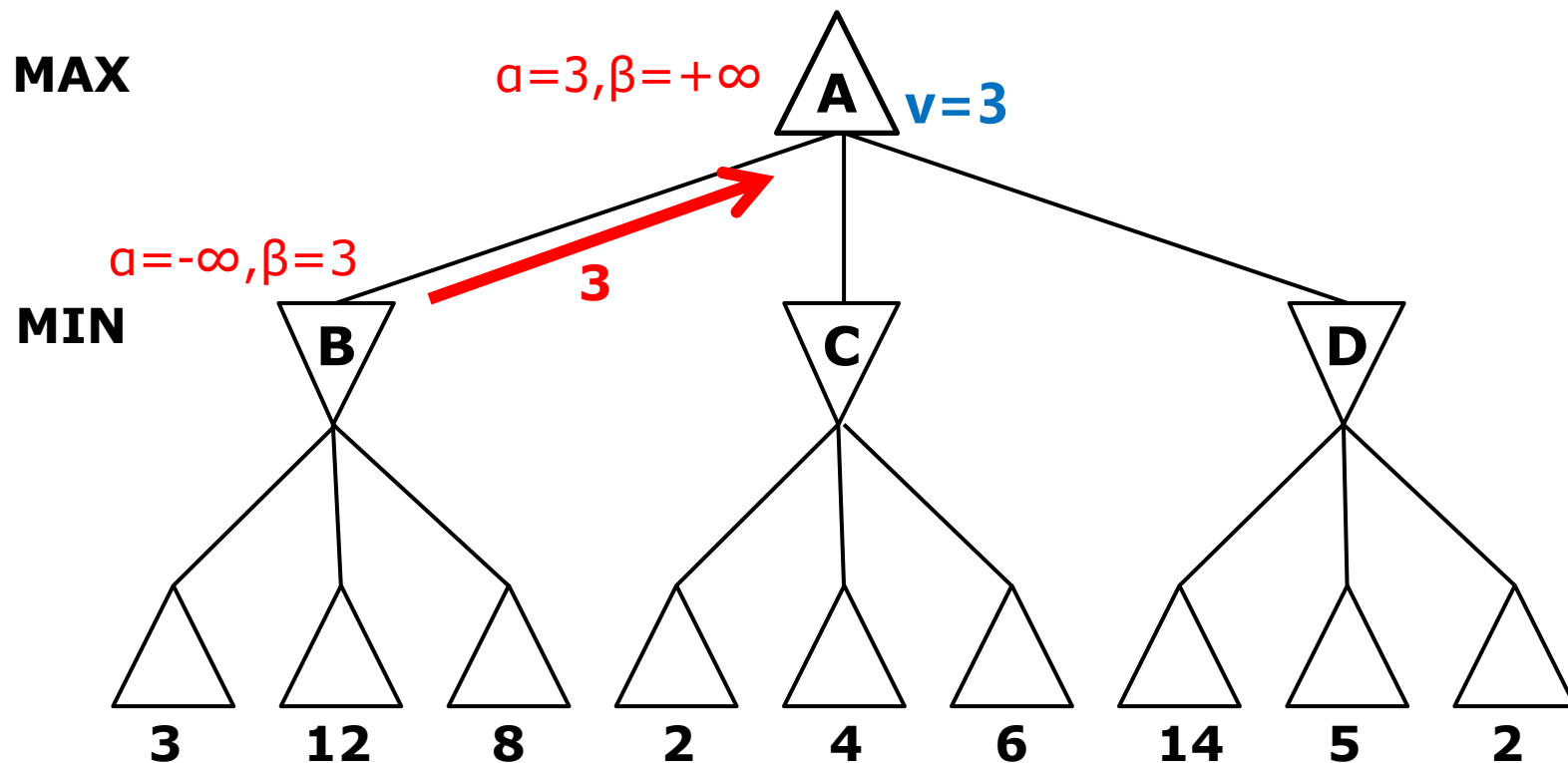


α = 到目前为止路径上MAX获得的最佳值（即极大值）

β = 到目前为止路径上MIN获得的最佳值（即极小值）

v = 当前结点所搜索过的子结点中的最佳值

α - β 剪枝

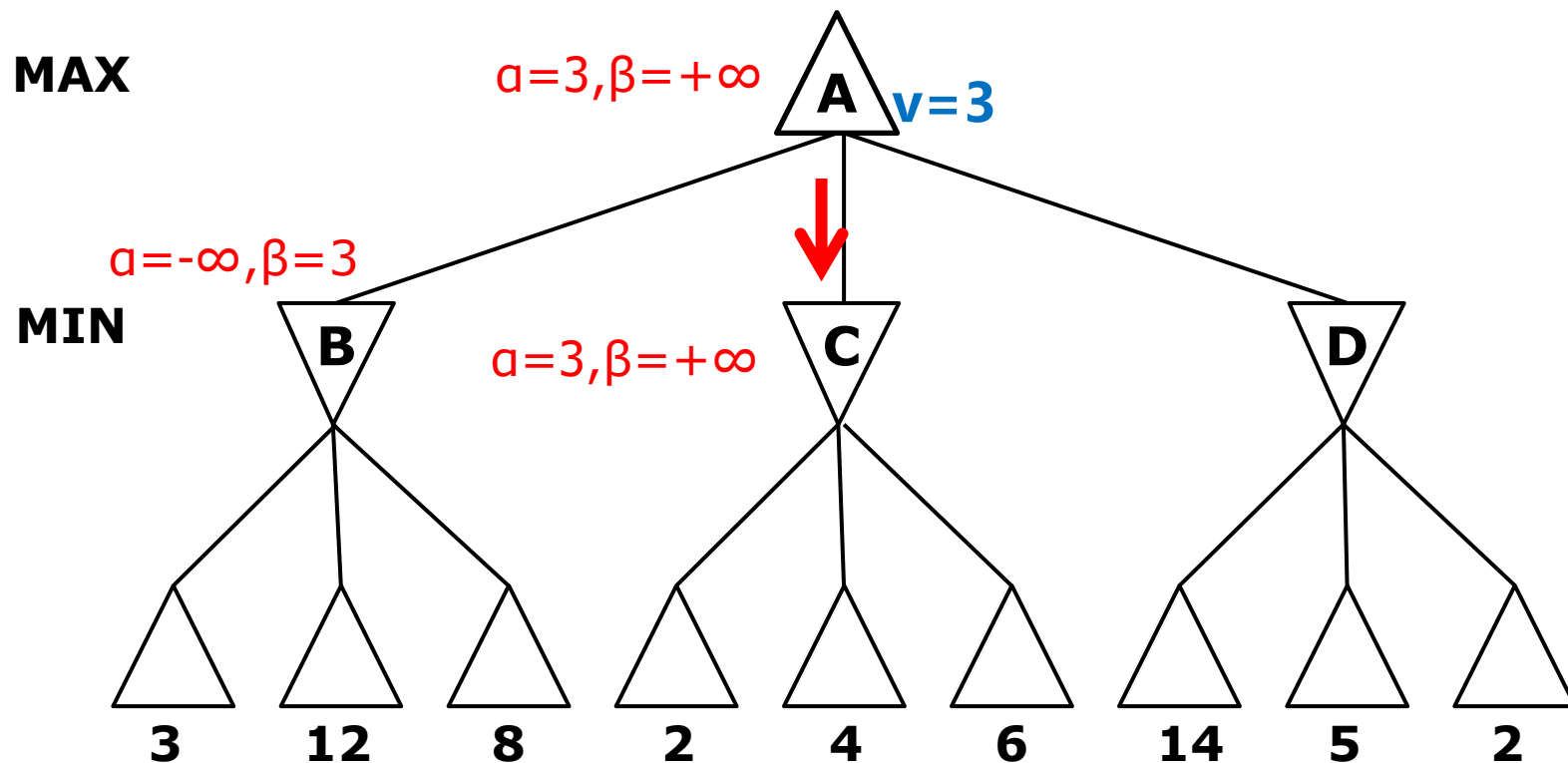


α = 到目前为止路径上MAX获得的最佳值（即极大值）

β = 到目前为止路径上MIN获得的最佳值（即极小值）

v = 当前结点所搜索过的子结点中的最佳值

α - β 剪枝



α = 到目前为止路径上MAX获得的最佳值（即极大值）

β = 到目前为止路径上MIN获得的最佳值（即极小值）

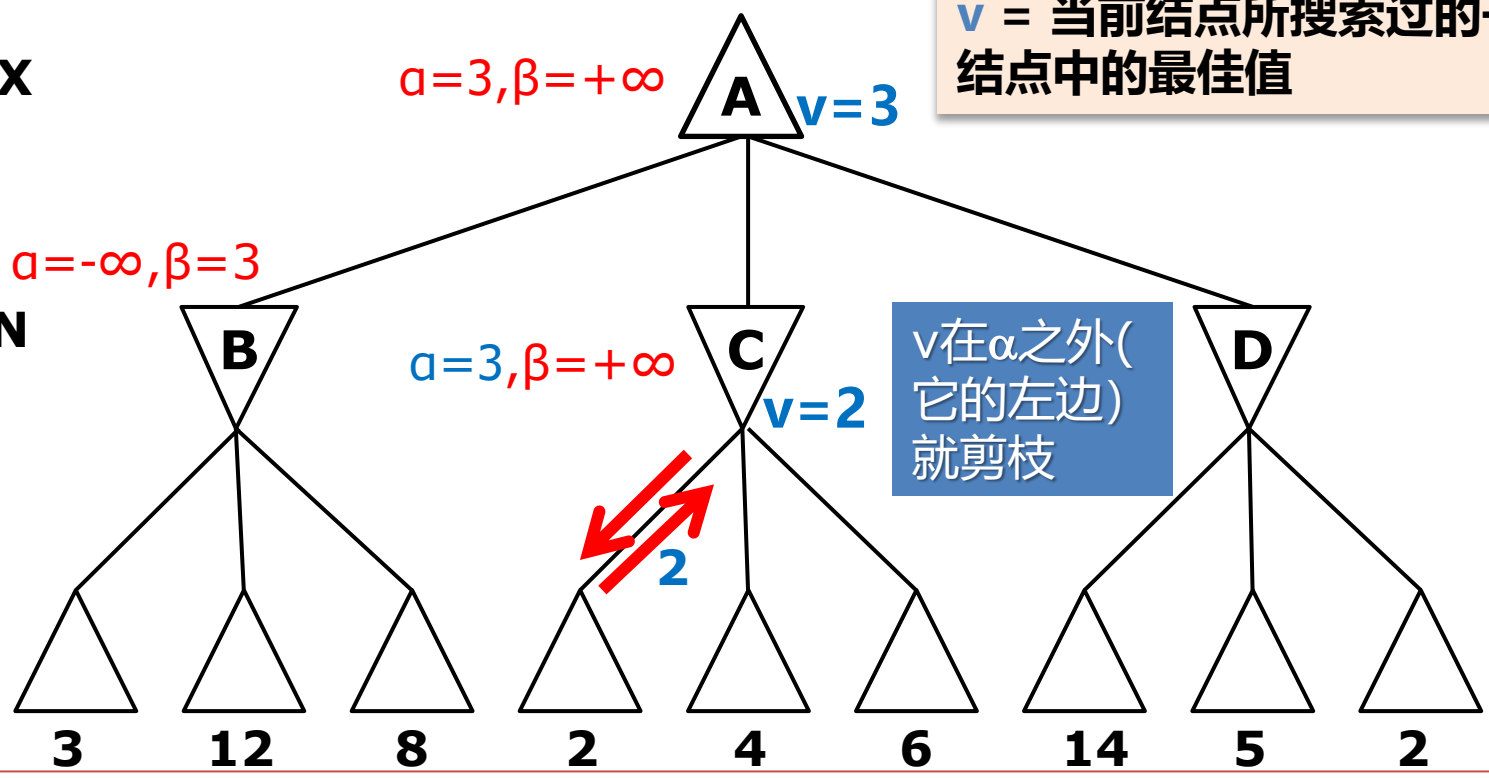
v = 当前结点所搜索过的子结点中的最佳值

α - β 剪枝

α = 到目前为止路径上MAX获得的最佳值 (即极大值)
 β = 到目前为止路径上MIN获得的最佳值 (即极小值)
 v = 当前结点所搜索过的子结点中的最佳值

MAX

MIN



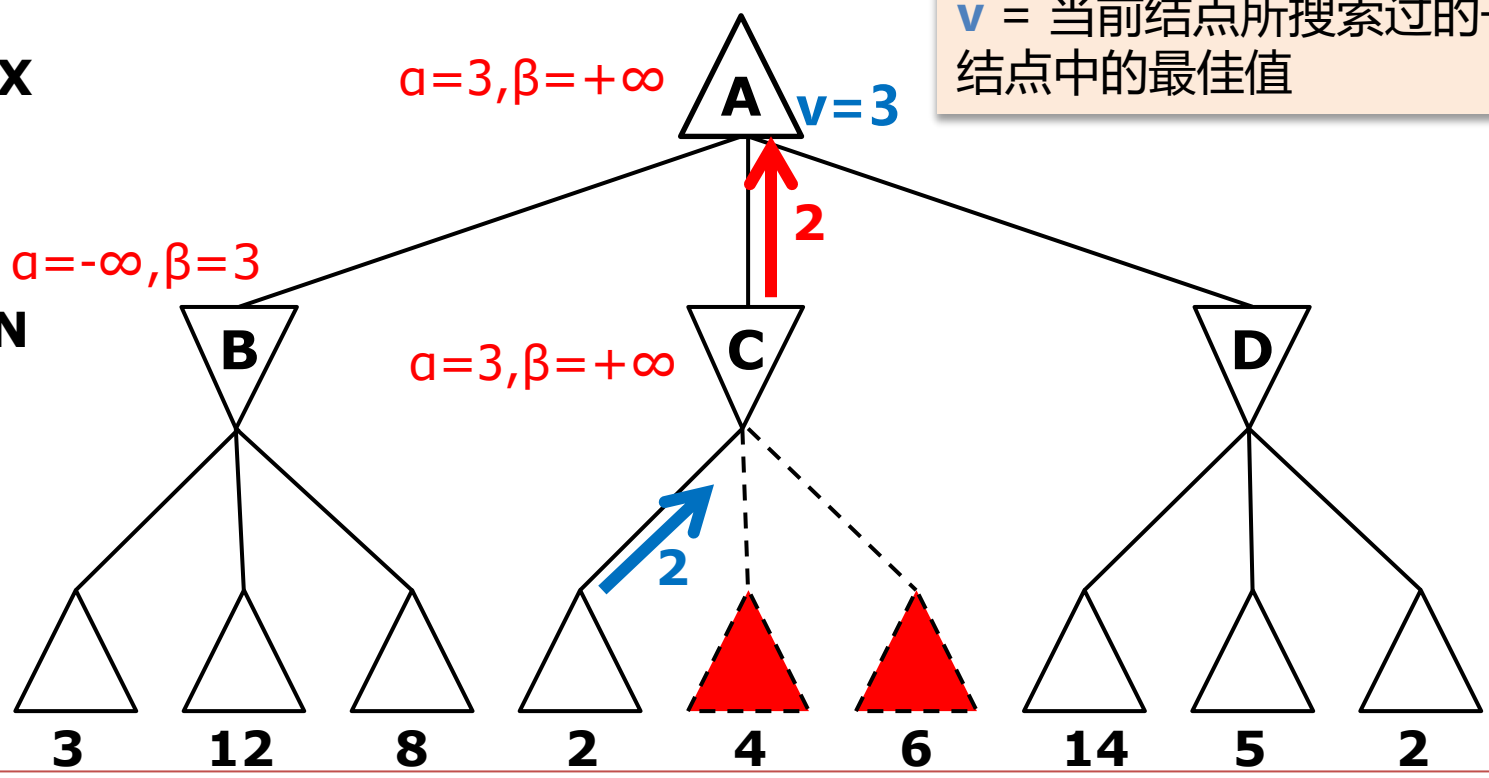
在min结点, 若当前搜索过的子结点的最小值 v 小于等于 α , 则剪去该min的其余子结点, 返回 v
 在max结点, 若当前搜索过的子结点的最大值 v 大于等于 β , 则剪去该max的其余子结点, 返回 v

α - β 剪枝

α = 到目前为止路径上MAX获得的最佳值 (即极大值)
 β = 到目前为止路径上MIN获得的最佳值 (即极小值)
 v = 当前结点所搜索过的子结点中的最佳值

MAX

MIN



在min结点, 若当前搜索过的子结点的最小值 v 小于等于 α , 则剪去该min的其余子结点, 返回 v
 在max结点, 若当前搜索过的子结点的最大值 v 大于等于 β , 则剪去该max的其余子结点, 返回 v

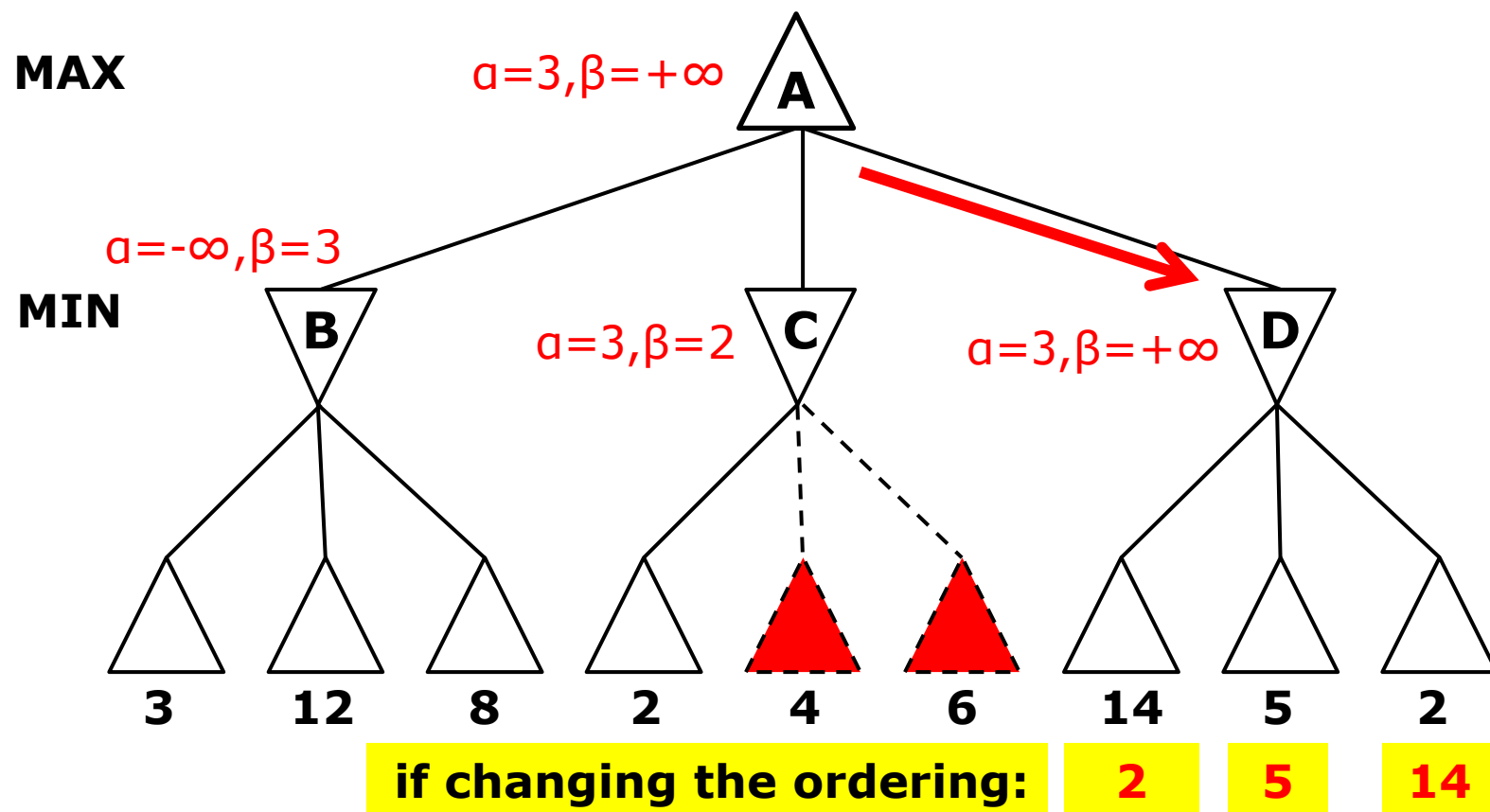
α - β 剪枝算法

function ALPHA-BETA-SEARCH($state$) **returns** an action
 $v \leftarrow \text{MAX-VALUE}(state, -\infty, +\infty)$
return the *action* in $\text{ACTIONS}(state)$ with value v

function MAX-VALUE($state, \alpha, \beta$) **returns** a *utility value*
if $\text{TERMINAL-TEST}(state)$ **then return** $\text{UTILITY}(state)$
 $v \leftarrow -\infty$
for each a **in** $\text{ACTIONS}(state)$ **do** 三次比较
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$
if $v \geq \beta$ **then return** v
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
return v

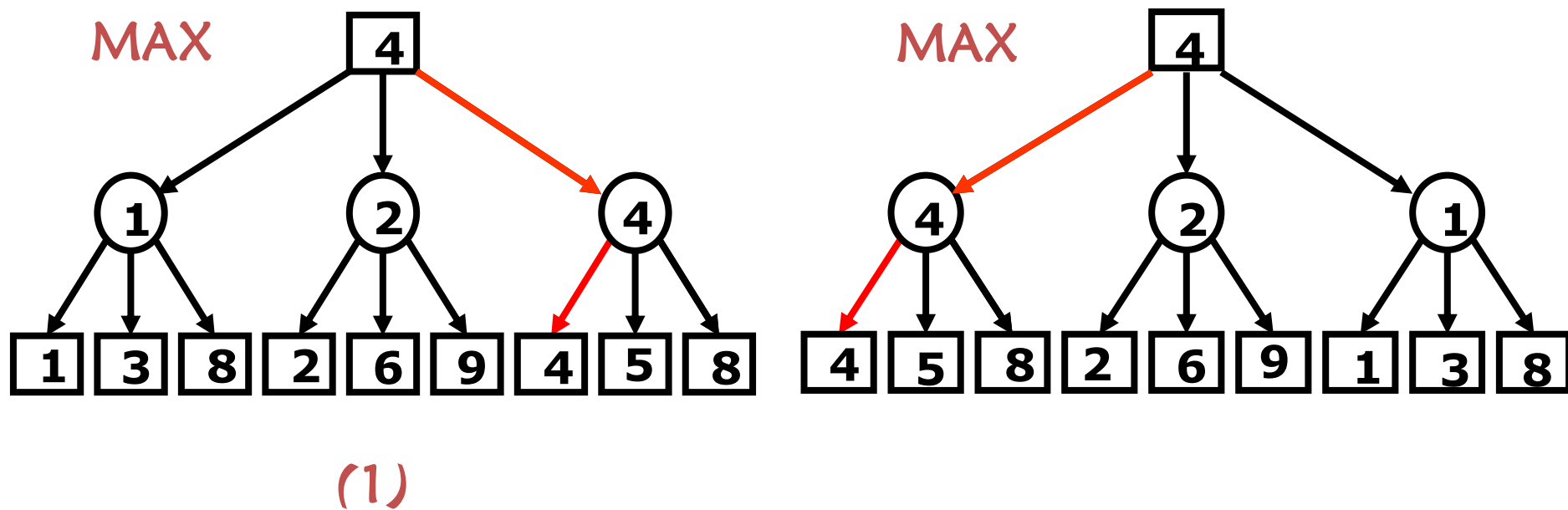
function MIN-VALUE($state, \alpha, \beta$) **returns** a *utility value*
if $\text{TERMINAL-TEST}(state)$ **then return** $\text{UTILITY}(state)$
 $v \leftarrow +\infty$
for each a **in** $\text{ACTIONS}(state)$ **do**
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$
if $v \leq \alpha$ **then return** v
 $\beta \leftarrow \text{MIN}(\beta, v)$
return v

考虑MIN的子结点的排序



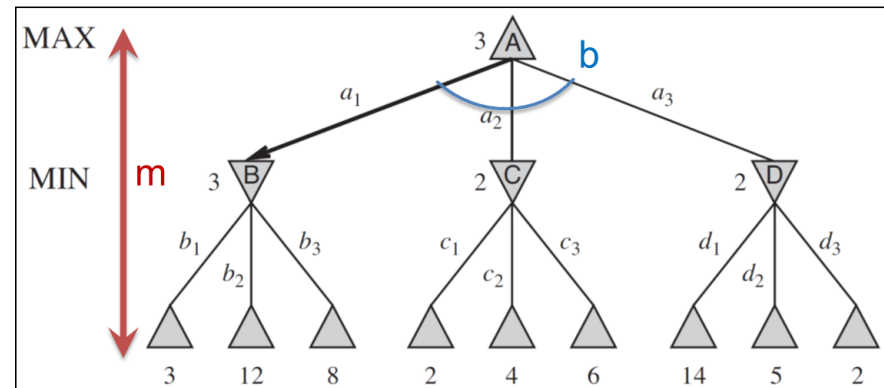
考虑MAX的子结点的排序

- find the best move:
- Which nodes are pruned?



α - β 算法剪枝的效果

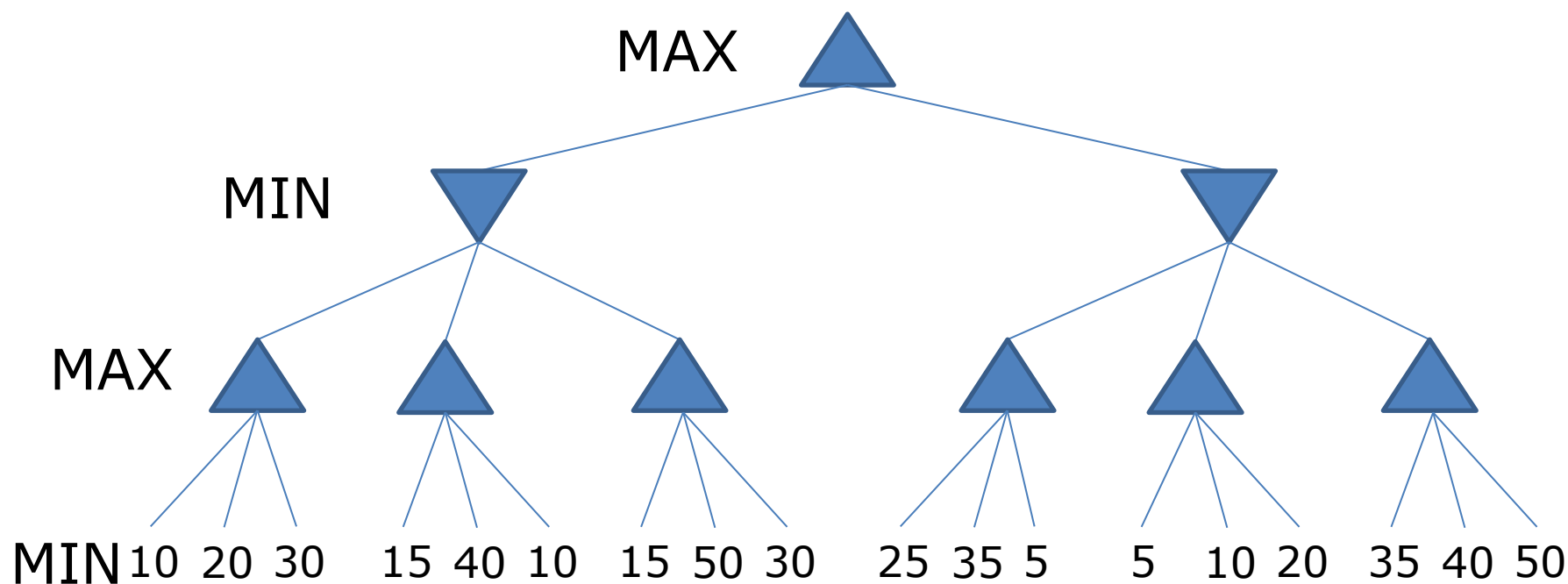
- 考虑分支因子都为 b 的博弈树，搜索深度为 m
- 最坏情况：需要检查检查 $O(b^m)$ 个结点，退化成Minimax算法
- 最好情况：只需检查 $O(b^{m/2})$ 个结点，如果
 - MAX 结点的MIN子结点按回传值的降序排列
 - MIN结点的MAX子结点按回传值的升序排列
- 一般情况：如果结点随机排序， 平均需要检查 $O(b^{3m/4})$ 个结点





练习： α - β 剪枝算法

- 对于下面博弈搜索树，采取Alpha-Beta剪枝，哪些结点会被剪掉？假设子结点按照从左到右的顺序访问。



能够解释区分完美决策与不完美的实时决策

效用函数和终止测试不再可用。

4 不完美的实时决策



博弈中的搜索

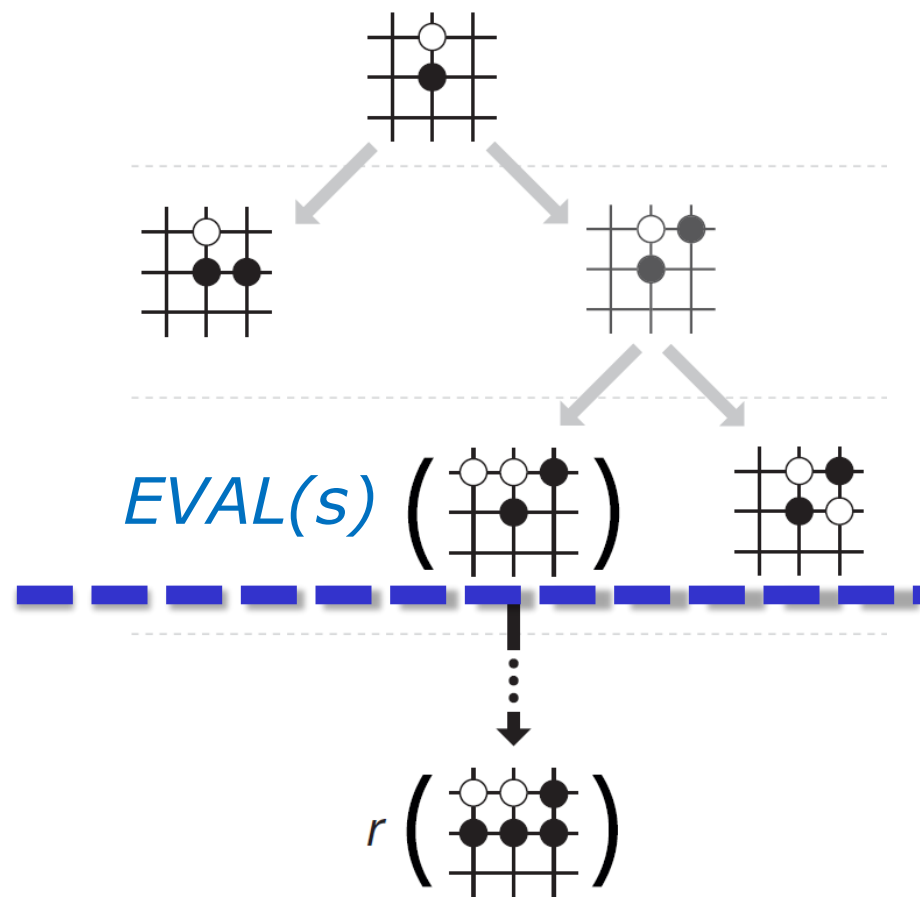
考虑有限的时间和空间

- 对手Agent (Min)和我们的Agent (Max)对抗
- Agent考虑自己的行动时必须考虑对手的可能行动
- 对手的行动未知
- 决策的时间和空间资源有限
- 状态空间巨大，有限时间内只能考虑很小的一部分

考虑时间空间限制

不完美的决策

- 要在给定的时间内决策，必须尽早**截断搜索**，不一定要到终止状态才能评估和回传结果
- 使用**启发式评估函数**对截断处的非终止状态进行评估，回传结果



评估函数、截断测试

- 用估计棋局效用值的启发式**评估函数**EVAL取代效用函数Utility，用**截断测试(cutoff test)**取代终止测试(Terminal Test)

$$\begin{aligned}
 \text{MINMAX}(s) = & \\
 \left(\begin{array}{ll}
 \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\
 \max_{a \in \text{Actions}(s)} \text{MINMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\
 \min_{a \in \text{Actions}(s)} \text{MINMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN}
 \end{array} \right.
 \end{aligned}$$

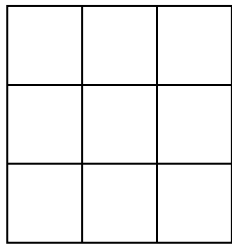
$$\begin{aligned}
 \text{H-MINMAX}(s, d) = & \\
 \left(\begin{array}{ll}
 \text{EVAL}(s) & \text{if } \text{CUTOFF-TEST}(s, d) \\
 \max_{a \in \text{Actions}(s)} \text{H-MINMAX}(\text{RESULT}(s, a), d+1) & \text{if } \text{PLAYER}(s) = \text{MAX} \\
 \min_{a \in \text{Actions}(s)} \text{H-MINMAX}(\text{RESULT}(s, a), d+1) & \text{if } \text{PLAYER}(s) = \text{MIN}
 \end{array} \right.
 \end{aligned}$$

评估函数(Evaluation Function)

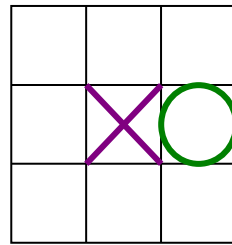
- 函数 $\text{EVAL}(s)$: 对于给定的棋局 s , 返回对博弈的期望效用值的估计
- 对棋局(状态) s 对于MAX有多好的启发式估计
 - $\text{EVAL}(s) > 0$: 有利于MAX, 值越大越对MAX有利
 - $\text{EVAL}(s) < 0$: 有利于MIN, 值越小越对MIN有利
 - $\text{EVAL}(s) = 0$: 中立

示例：井字棋(tic-tac-toe)

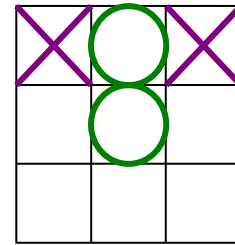
□ $EVAL(s) =$ (number of rows, columns, and diagonals open for MAX) - (number of rows, columns, and diagonals open for MIN)



$$8 - 8 = 0$$

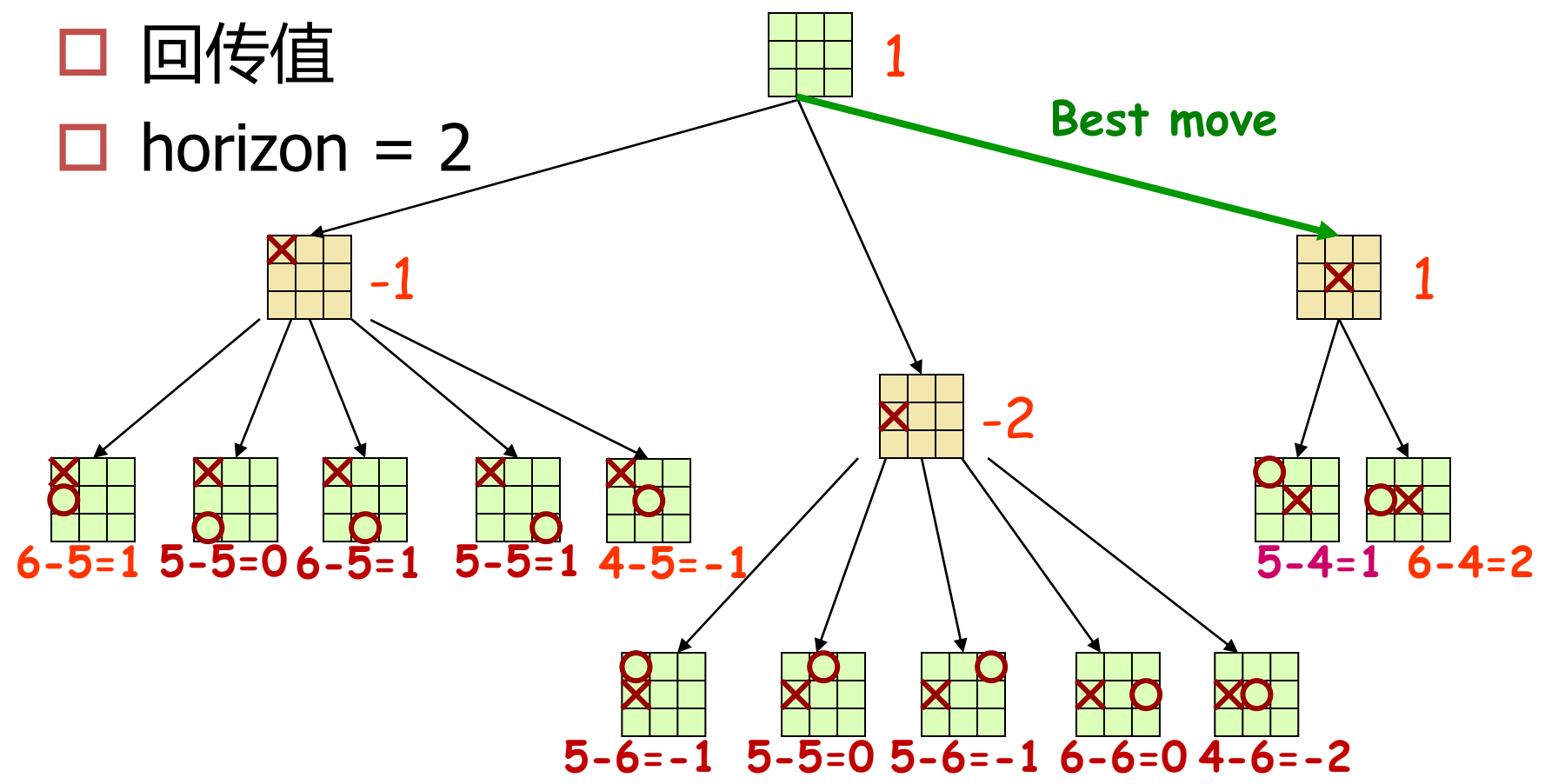


$$6 - 4 = 2$$



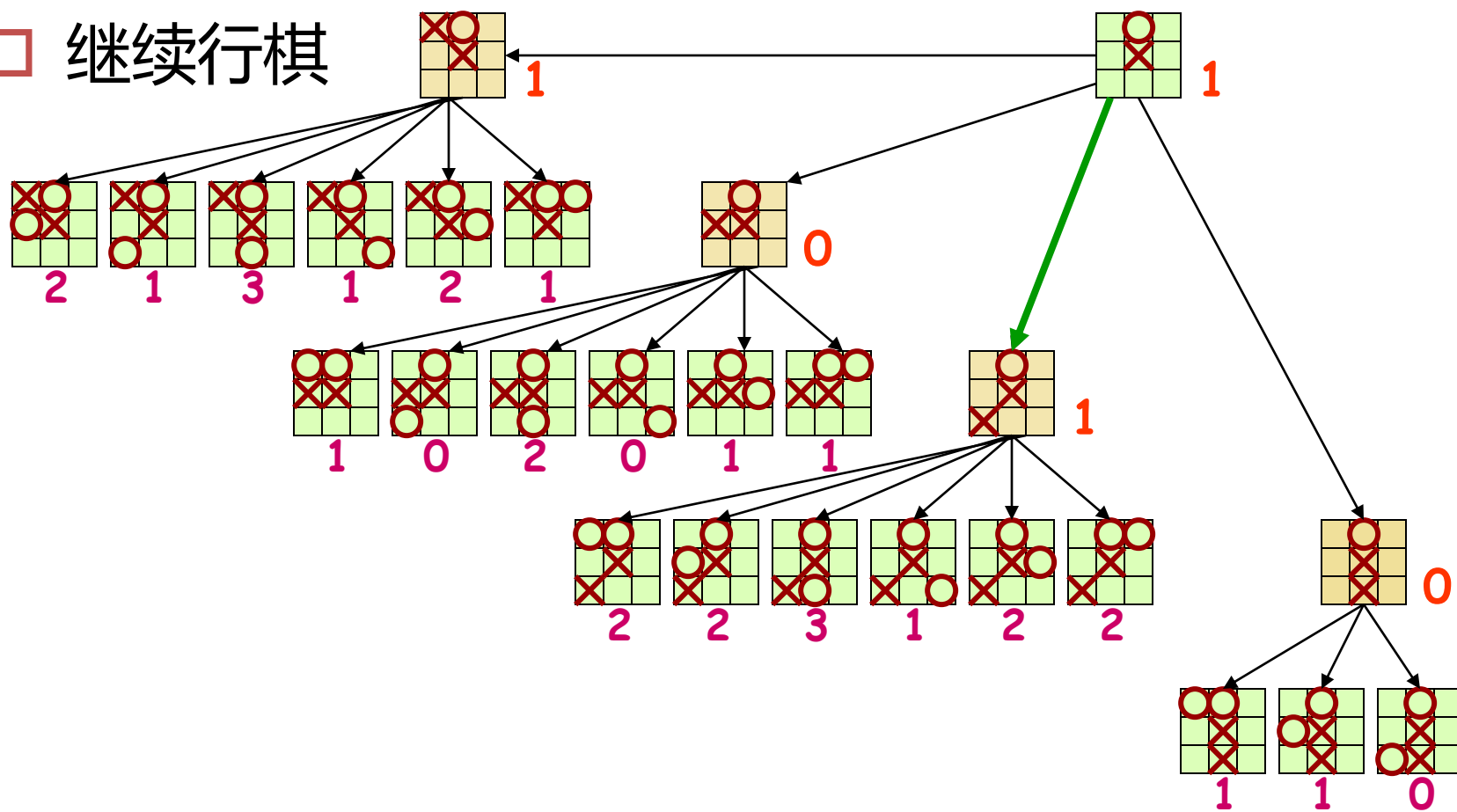
$$3 - 3 = 0$$

示例：井字棋(tic-tac-toe)



示例：井字棋(tic-tac-toe)

□ 继续行棋



如何截断

- 如何截断，以满足时间限制？
- 直接的方法：设置固定的深度限制，深度值设定必须符合游戏规则许可的时间。
- 更好的方法：迭代加深，直到时间用完。

迭代加深

□ 迭代加深

■ $1+b+b^2+\dots+b^m=(b^{m+1}-1)/(b-1)=O(b^m)$

□ 一层层展开，一层层搜索，因为“穷尽”而没有风险，不会漏掉展开深度内的最优解

□ 假设计算机搜索节点速率为1M/秒，中国象棋 $b=45$ ($b=40\sim 50$)，在不同给定时间内达到的搜索层数：

时间限制	1秒	1分	1小时	1天	1年	10年	100年
搜索层数	3.63	4.70	5.78	6.62	8.17	8.77	9.36

向前剪枝

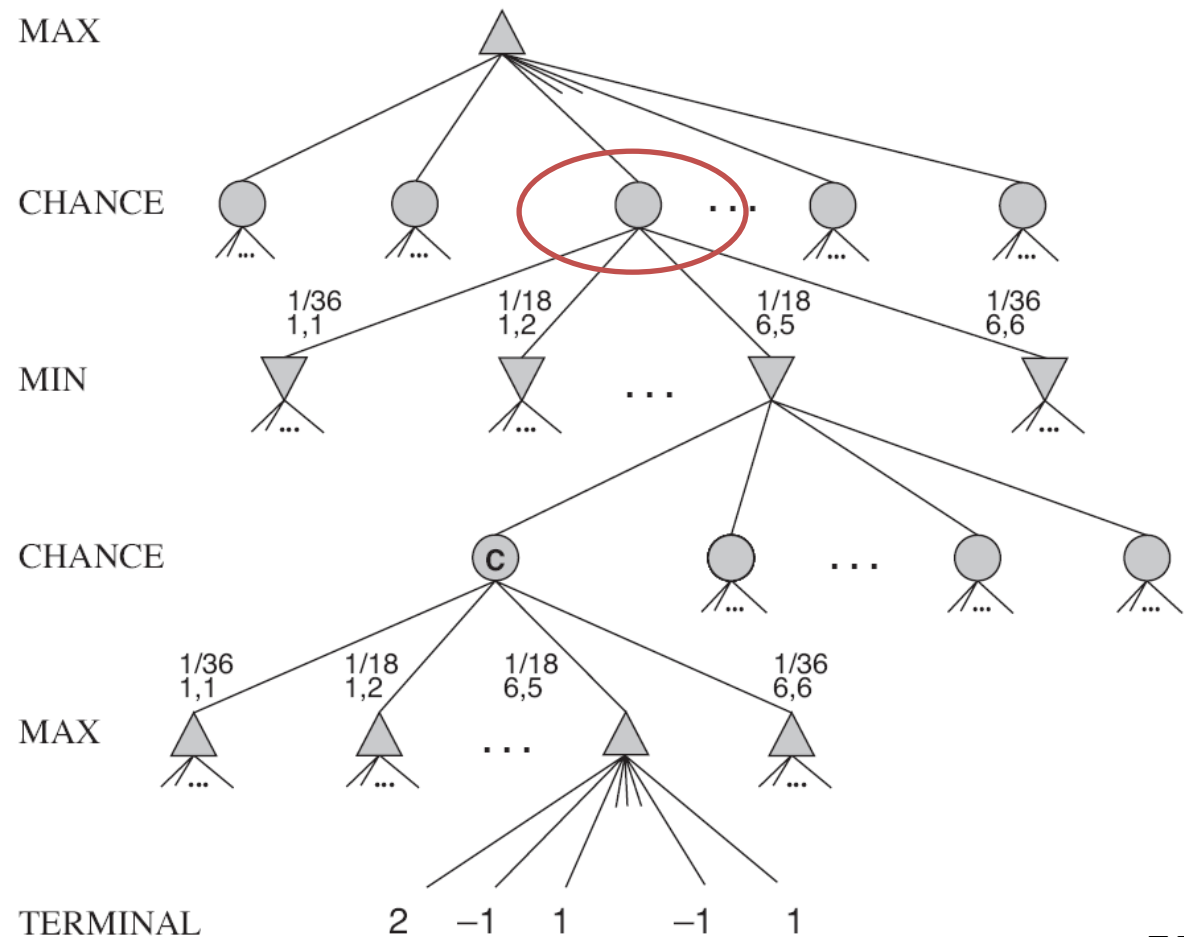
- 除了截断搜索，还可使用**向前剪枝**。大多数人在下象棋的时候，对每个棋局**只考虑部分行棋**。
- 一种向前剪枝的方法是**柱搜索**：在每一层，**只考虑最好的n步行棋可能**（称为“柱”），并不是考虑所有行棋招数。但无法保证最佳的行棋不被裁剪掉。

5 随机博弈



带机会结点的博弈树

□ 机会结点(chance nodes)



期望值(expected value):所有可能结果的平均值

期望极小极大值

- 把确定性博弈中的极小极大值一般化为期望极小极大值(expectiminimax value)

EXPECTIMINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \\ \sum_r P(r) \text{EXPECTIMINIMAX}(\text{RESULT}(s, r)) & \text{if } \text{PLAYER}(s) = \text{CHANCE} \end{cases}$$

本章总结

- 极小极大算法是博弈的基本算法
- α - β 搜索算法可以计算出和极小极大算法一样的最优招数，通过剪枝，效率更高
- α - β 算法跳过了显然不好的情况
- 通常考虑整棵博弈树不可行，需要截断搜索，应用启发式评估函数对状态的进行评估
- 随机博弈问题，搜索树中引入机会结点，计算期望极小极大值

Thanks!

next: CSP